

Arboles de clasificación

Maria Laura Vranic

Árboles

Árboles de clasificación

Árboles de decisión

Árboles de regresión



🌳 1. Árbol de decisión (Decision Tree)

Es el **término general** que describe la estructura del modelo: un árbol con nodos de decisión y hojas que representan resultados. Se usa tanto para clasificación como para regresión.

🧭 2. Árbol de clasificación (Classification Tree)

- Se usa cuando la **variable objetivo es categórica** (por ejemplo, "apto/no apto", "tipo de alimento", "contaminado/no contaminado").
- **Ejemplo**: Predecir si un alimento está contaminado según sus características químicas.
- Las hojas del árbol contienen **clases**.

📈 3. Árbol de regresión (Regression Tree)

- se usa cuando la **variable objetivo es continua** (por ejemplo, concentración de nitritos, temperatura, precio).
- **Ejemplo**: Predecir el nivel de nitritos en un embutido.
- Las hojas del árbol contienen **valores numéricos**.

Temas

¿Qué es un árbol de clasificación?

Entrenamiento de un árbol de clasificación con el algoritmo CART

Generación de predicciones con un árbol de clasificación

Árboles de clasificación en Scikit-Learn

Fácil de interpretar

¿Qué es un árbol de regresión?

Entrenamiento de un árbol de regresión con el algoritmo CART

Generación de predicciones con un árbol de regresión

Árboles de regresión en Scikit-Learn



Ajuste de hiperparámetros

Tiempo de entrenamiento largos

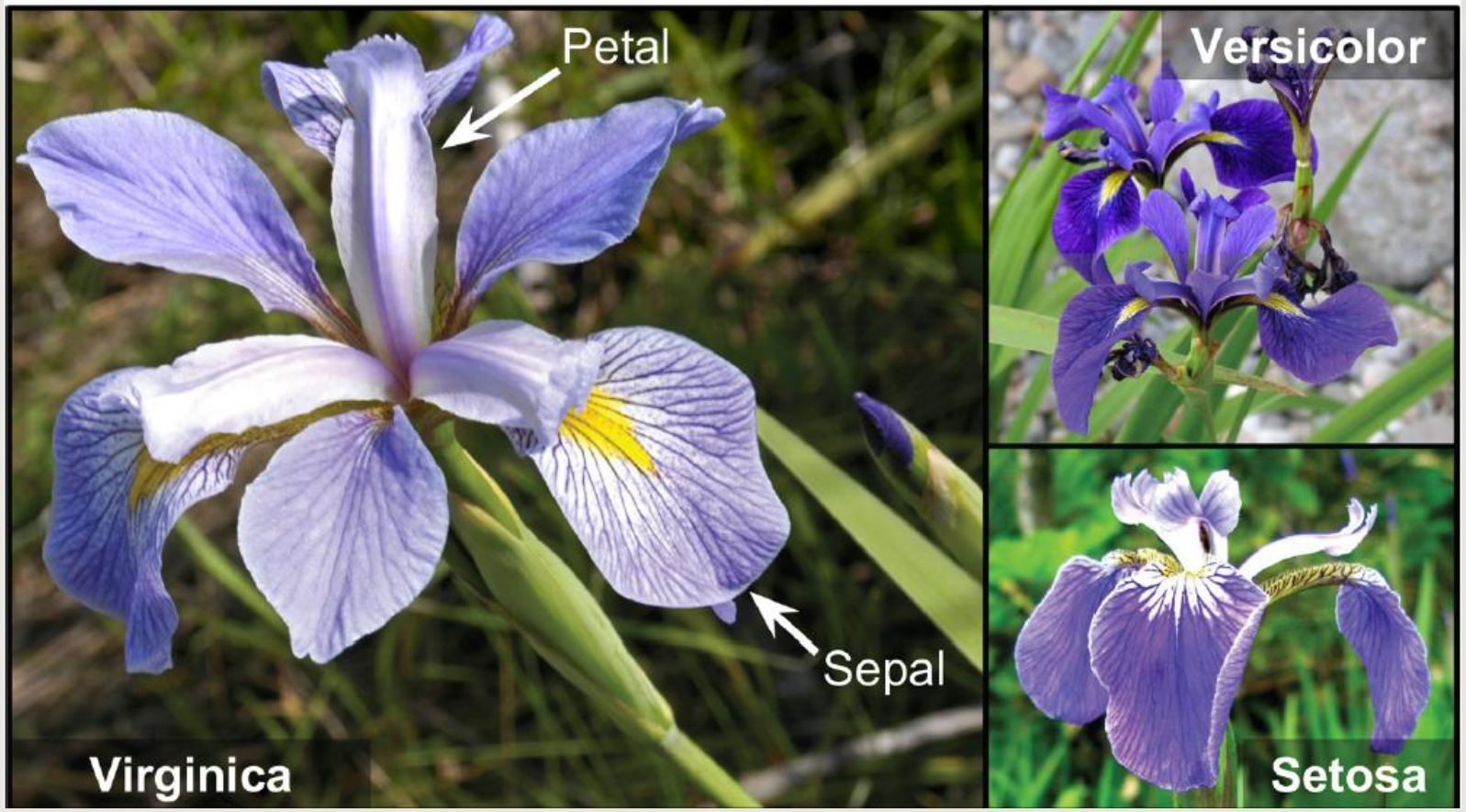
Entrenamiento y visualización de un árbol de decisión

Pasito a pasito…

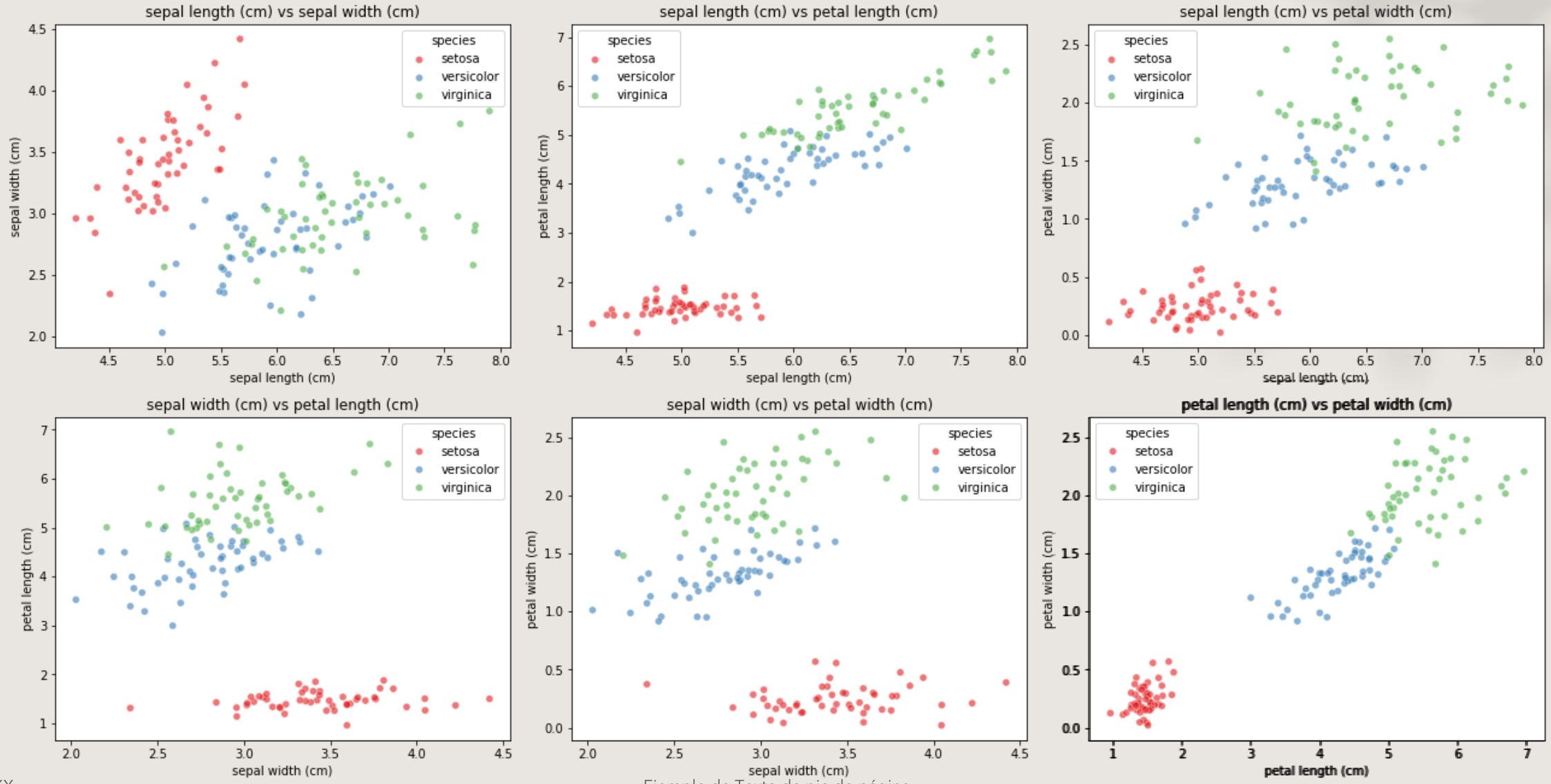


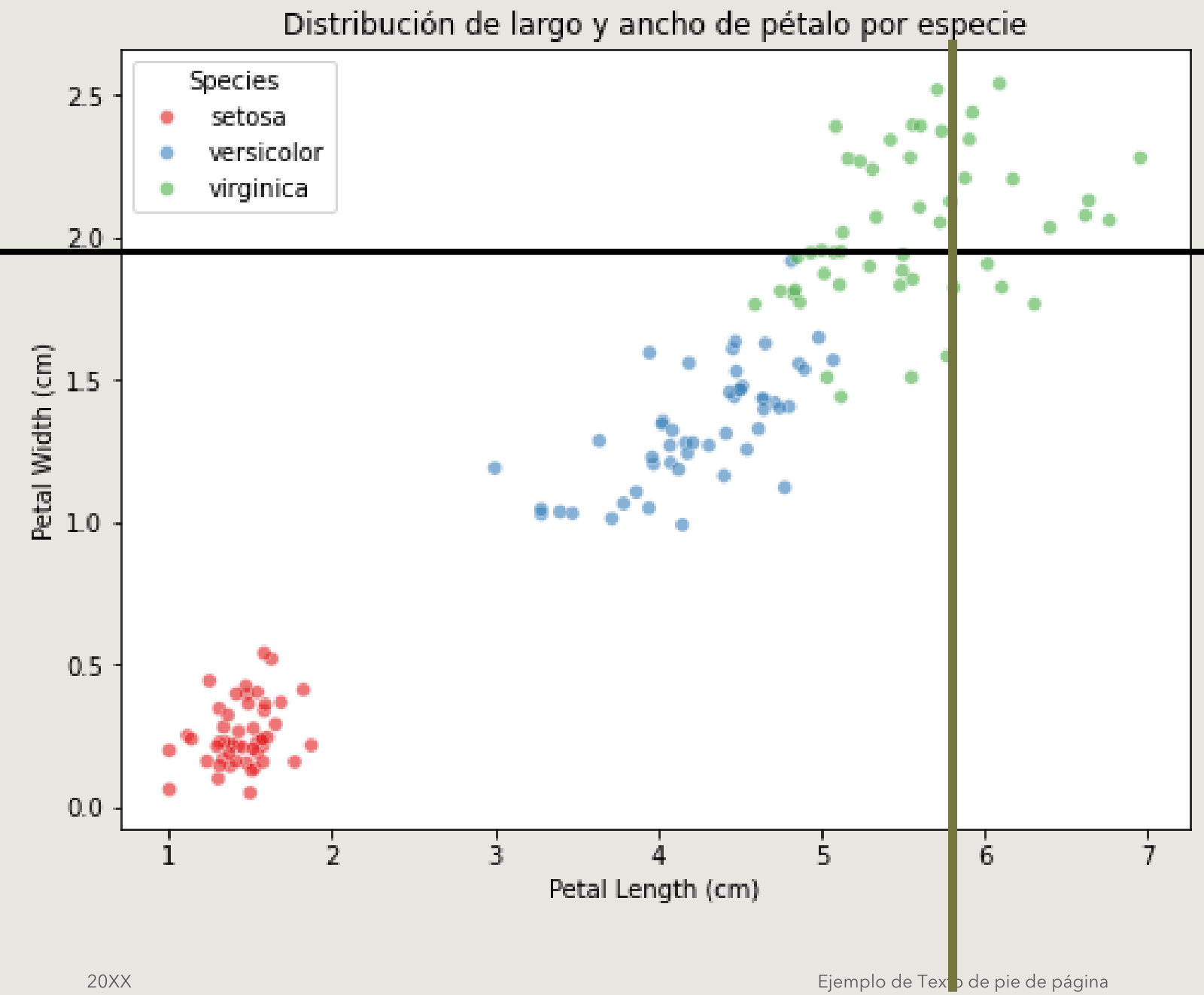


Data set: Iris



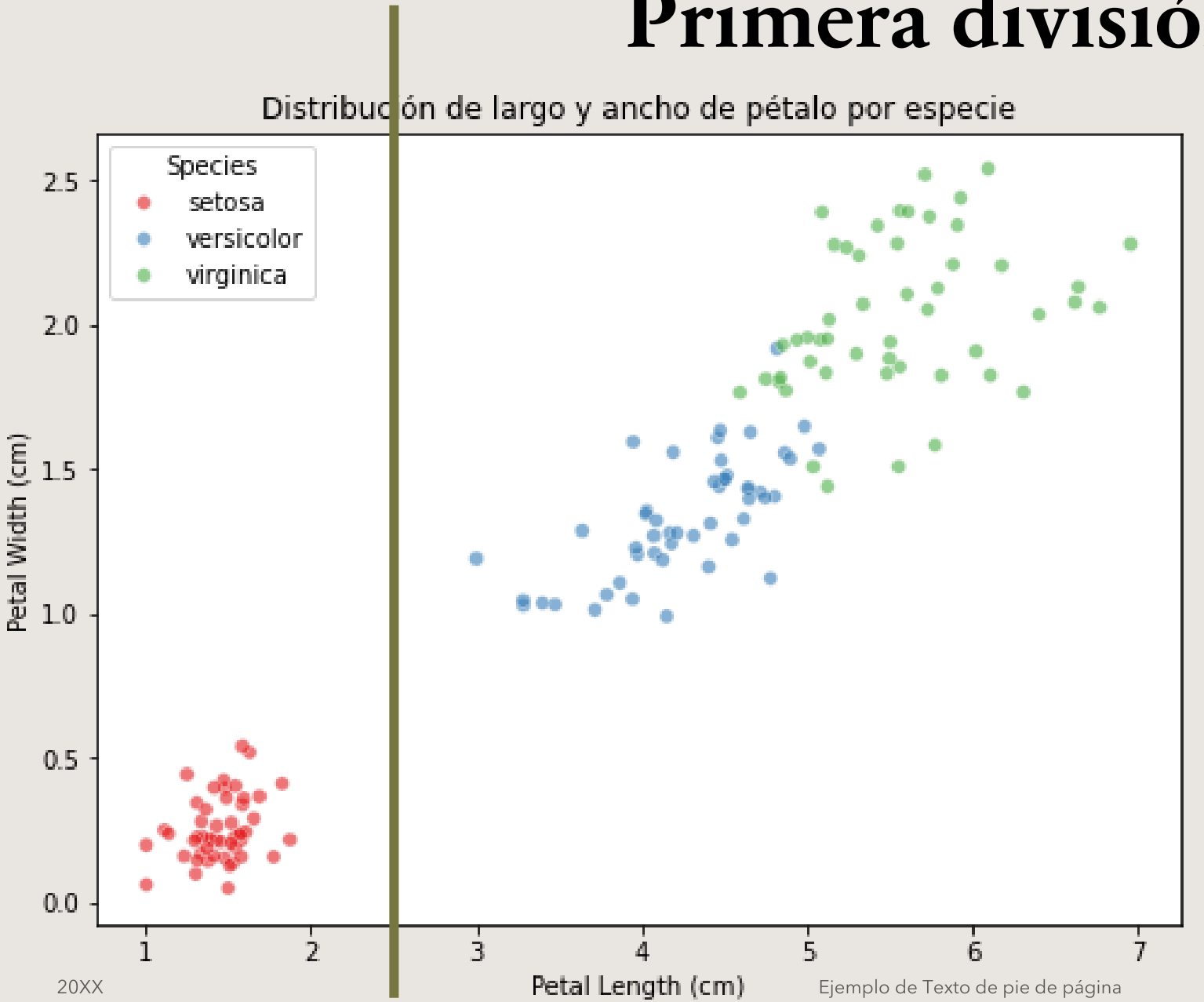
Gráficos de dispersión por pares de variables





- Si tuvieran que trazar una recta **vertical** (basada en el largo del pétalo) o **horizontal** (basada en el ancho del pétalo) para separar los grupos,
- ¿qué variable parece separar mejor las especies en este gráfico: el largo del pétalo o el ancho del pétalo?
- ¿Por qué?

Primera división

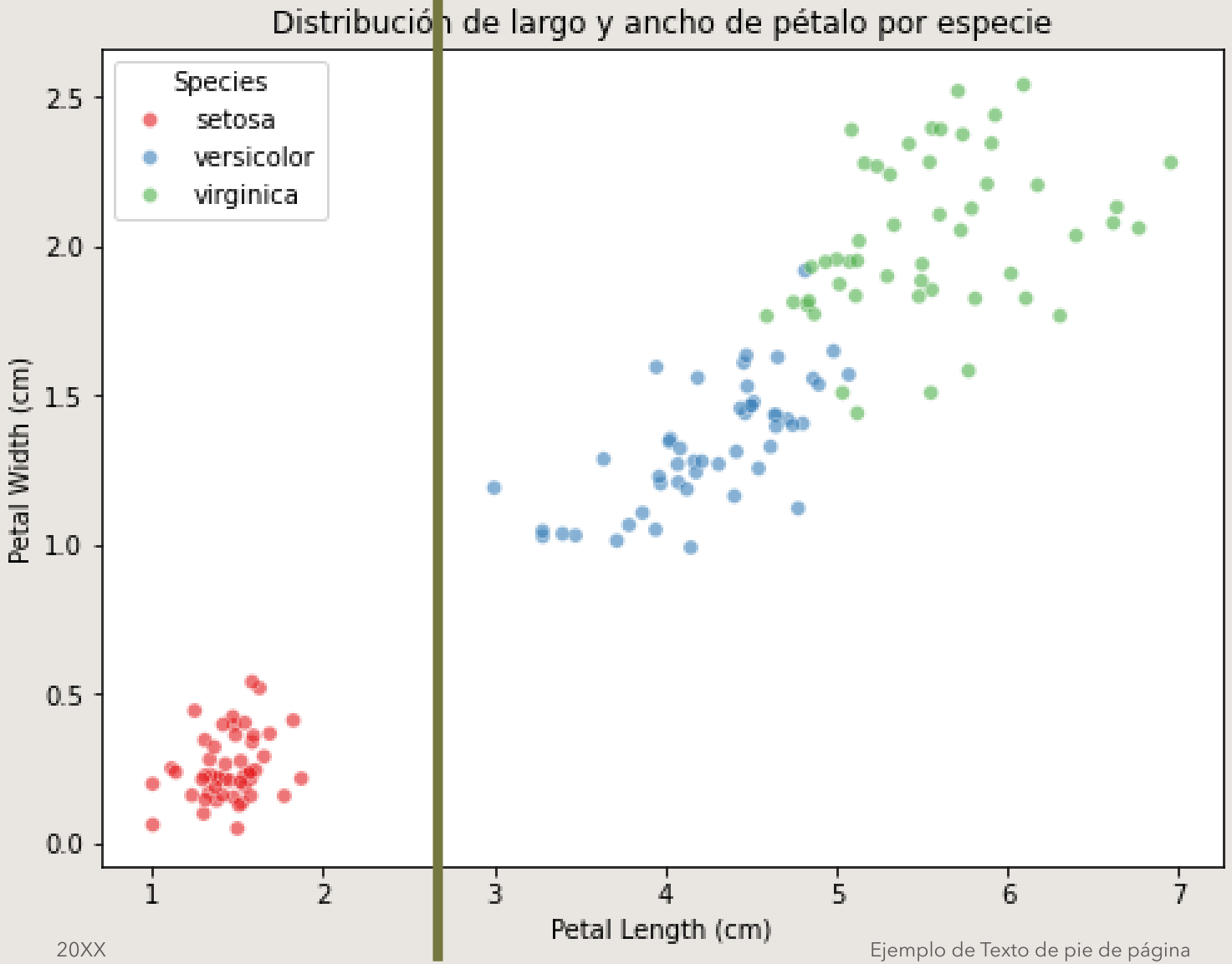


Si tuvieras que hacer una primera regla para separar *Setosa* del resto,

¿qué condición pondrías sobre el largo del pétalo?

(Por ejemplo: "Si el largo del pétalo < 2.5 cm → Setosa")

Hagamos una segunda división

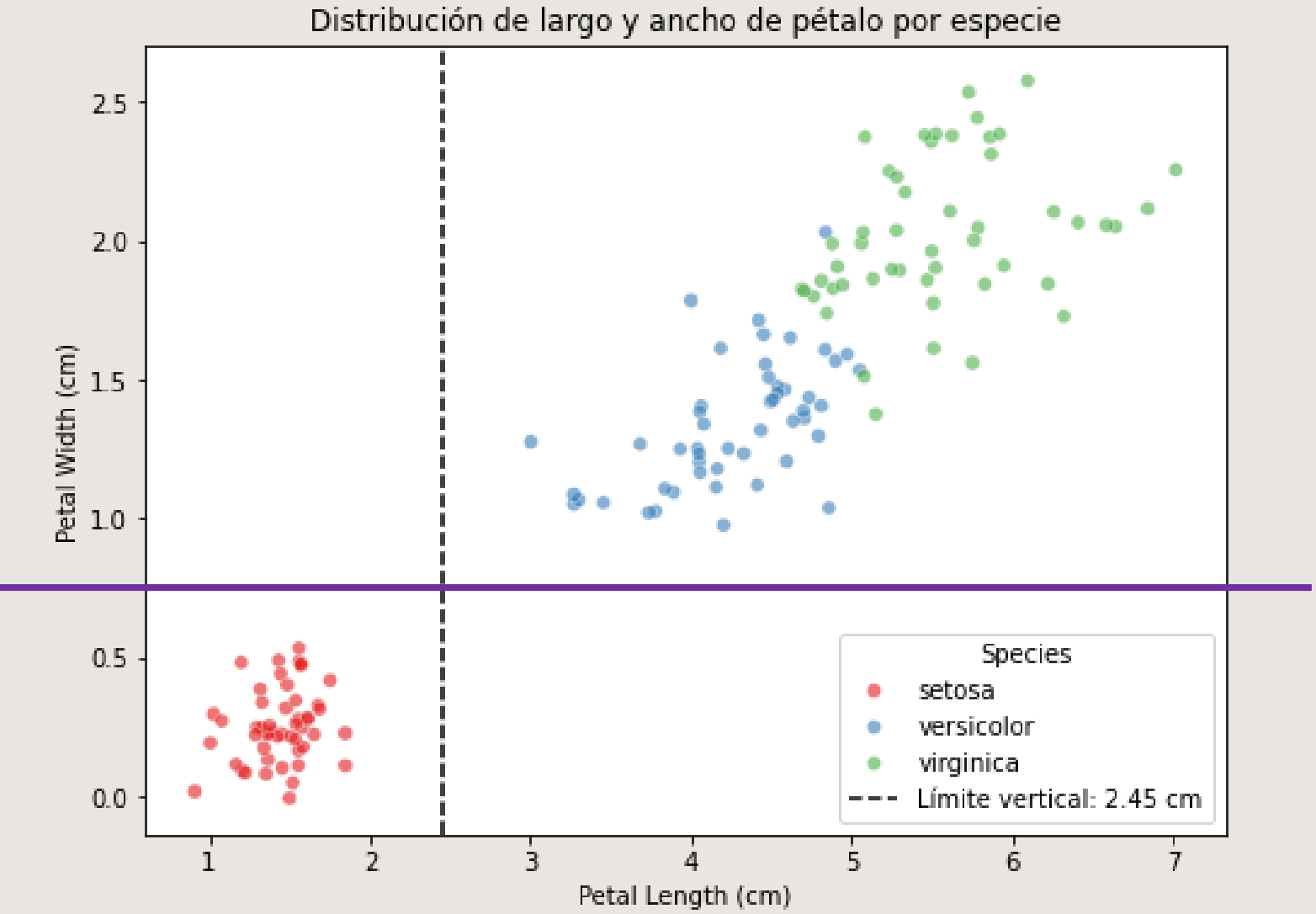


Después de separar *Setosa*, ¿cómo podrías dividir las otras dos especies (*Versicolor* y *Virginica*)?

¿Qué variable usarías ahora?

¿Cuál parece más útil?

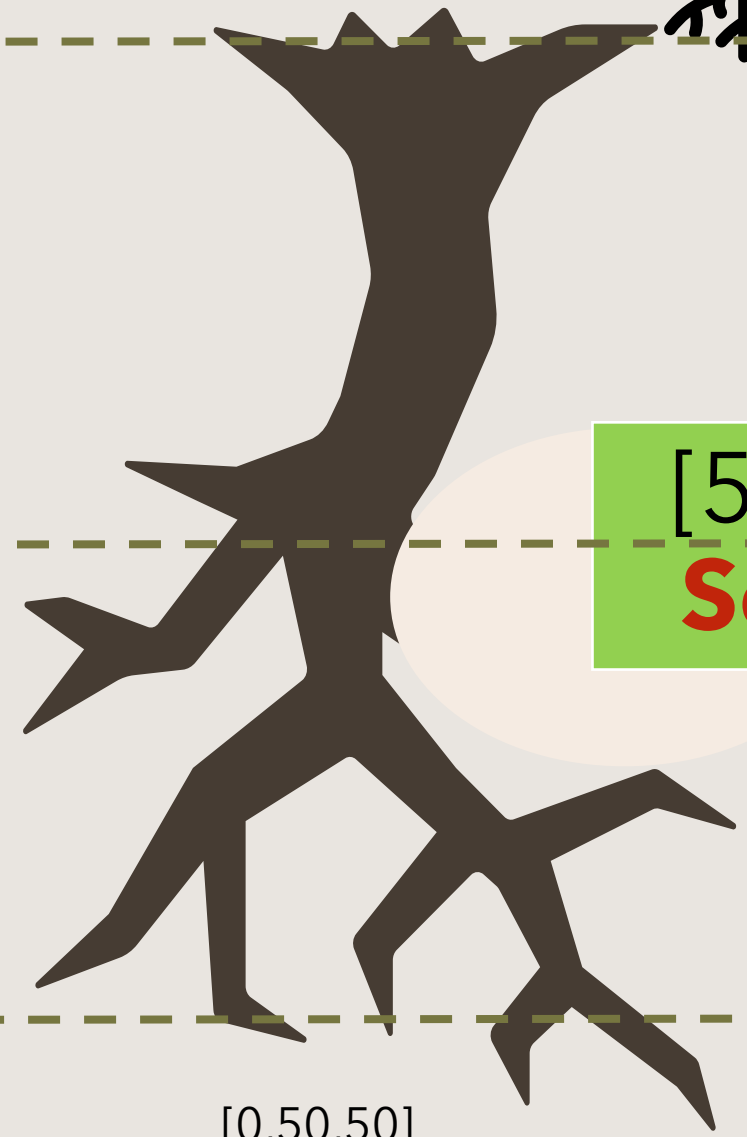
Hagamos una segunda división



Definición de umbrales

¿Qué valor aproximado del ancho del pétalo usarías para separar *Versicolor* de *Virginica*?

(Por ejemplo: "Si el ancho del pétalo < 1.75 cm → Versicolor, si no → Virginica")



Pethal length (cm)
<=2.45

Raíz

0

TRUE

FALSE

1ra Partición binaria

[50,0,0]
Setosa

Petal width (cm)
<= 1.75 [0,50,50]

Nodo interno

1

2da Partición binaria

[0,49,5]
Versicolor

[0,1,45]
Virginica

2

[0,50,50]
[#Setosa, #Versicolor,#Virginica]

Validación visual

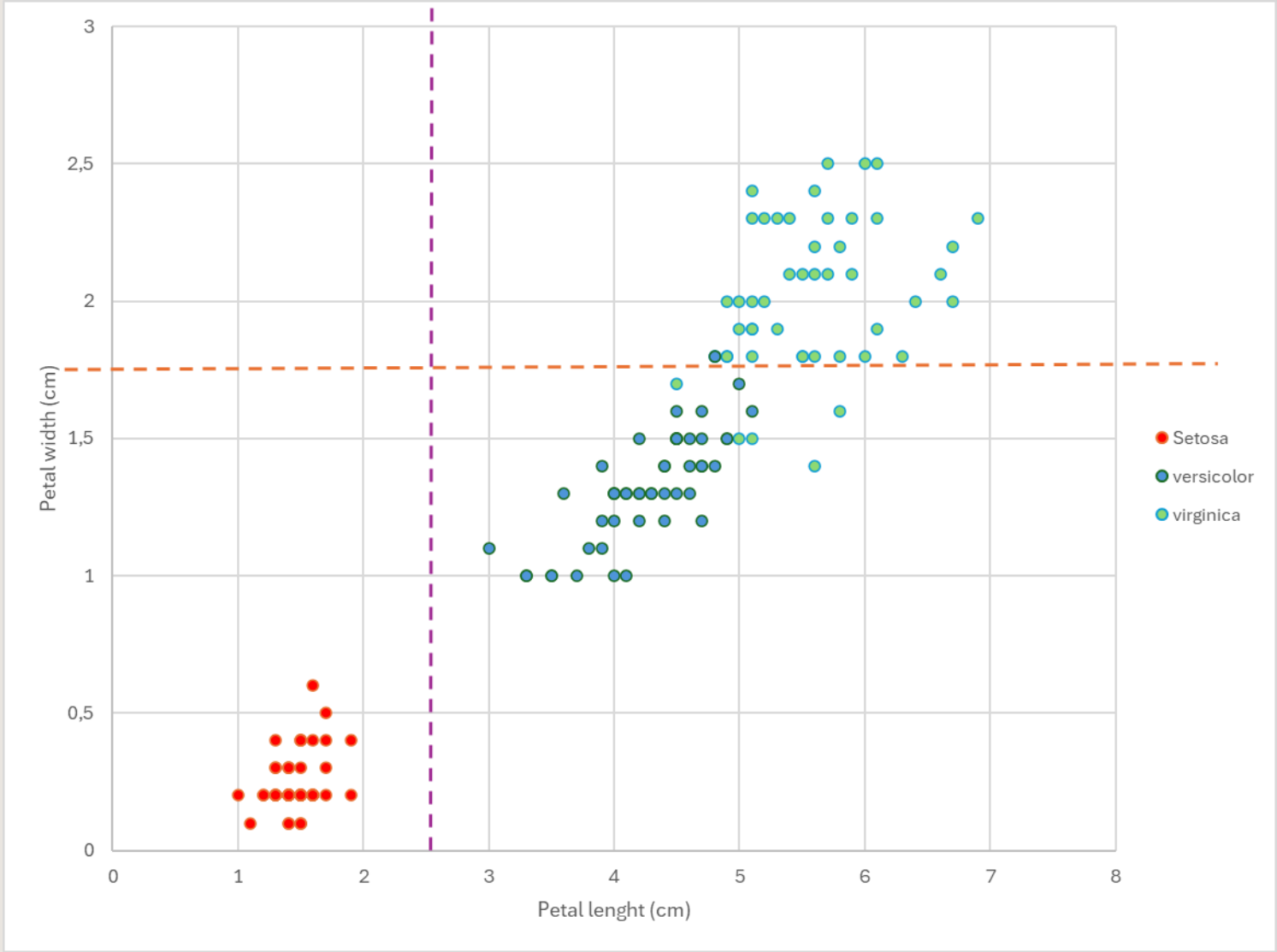
¿Tu árbol tiene errores?

¿Hay puntos que quedarían mal clasificados con esas reglas?

¿Cómo podrías ajustar los umbrales para mejorar la separación?

¿Qué ventajas tiene usar reglas simples basadas en variables?

¿Qué limitaciones ves si los datos estuvieran más mezclados?



Veamos cómo el árbol realiza predicciones

Pethal length (cm)
<=2.45

TRUE

FALSE

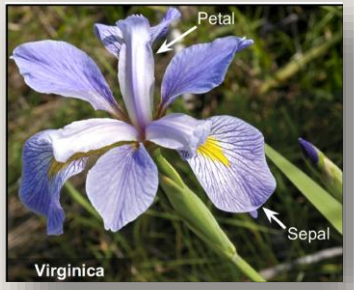


[50,0,0]
Setosa

Petal width (cm)
<= 1.75 [0,50,50]

[0,49,5]
Versicolor

[0,1,45]
Virginica







¿Cómo se construye automáticamente un árbol de clasificación?

CART

Índice Gini (grado de impureza de un nodo, G_i)

k: $número\ de\ clases$, 3

$$G_i = 1 - \sum_{k=1}^{K} p_{i,k}^2$$

i $número\ de\ nodos$, 2

$p_{i,k}$ $es\ la\ proporción\ de\ instancias\ de\ la\ clase\ k$
$entre\ las\ instancias\ de\ entrenamiento\ en\ el\ nodo\ j$.

Interpretación:

Si todas las observaciones pertenecen a una sola clase → $p_i = 1 \rightarrow Gini = 0$ (nodo puro)
Si las clases están distribuidas uniformemente → el Gini es máximo (más impuro).

Índice Gini: Rango de valores

- **Mínimo = 0**
  - Nodo **puro**: todos los elementos pertenecen a la misma clase.
- **Máximo** depende del número de clases:

  Para **2 clases** (binario):

- $Gini_{max} = 0.5$ (las dos clases están en proporción 50%-50%).

  Para **k clases**:

- $Gini_{max} = 1 - \frac{1}{k}$ (todas las clases están igualmente representadas).

Ejemplos

- Nodo con 100% de una clase → Gini = 0.
- Nodo con 50% clase A y 50% clase B → Gini = 0.5.
- Nodo con 3 clases iguales (33% cada una) →
- $Gini = 1 - (0.33^2 + 0.33^2 + 0.33^2) \approx 0.67$

$$G_i = 1 - \sum_{k=1}^{K} p_{i,k}^2$$

Pethal length (cm) <=2.45

$$G_0 = 1 - \left(\frac{50}{150}\right)^2 - \left(\frac{50}{150}\right)^2 - \left(\frac{50}{150}\right)^2 = 0{,}664$$

TRUE **FALSE**

[50,0,0]
Setosa

Petal width (cm) <= 1.75
[0,50,50]

$$G_{1right} = 1 - \left(\frac{50}{50}\right)^2 - \left(\frac{0}{50}\right)^2 - \left(\frac{0}{50}\right)^2 = 0$$

$$G_{1left} = 1 - \left(\frac{0}{100}\right)^2 - \left(\frac{50}{100}\right)^2 - \left(\frac{50}{100}\right)^2 = 0.5$$

[0,49,5]
Versicolor

[0,1,45]
Virginica

$$G_{2left} = 1 - \left(\frac{0}{54}\right)^2 - \left(\frac{49}{54}\right)^2 - \left(\frac{5}{54}\right)^2 = 0{,}168$$

$$G_{2right} = 1 - \left(\frac{0}{46}\right)^2 - \left(\frac{1}{46}\right)^2 - \left(\frac{45}{46}\right)^2 = 0.043$$

$$G_i = 1 - \sum_{k=1}^{K} p_{i,k}^2$$

$$G_i = 1 - \sum_{k=1}^{K} p_{i,k}^2$$

Pethal length (cm) <=2.45

$$G_0 = 1 - \left(\frac{50}{150}\right)^2 - \left(\frac{50}{150}\right)^2 - \left(\frac{50}{150}\right)^2 = 0{,}664$$

[50,0,0]
Setosa

Petal width (cm) <= 1.75
[0,50,50]

$$G_{1right} = 1 - \left(\frac{50}{50}\right)^2 - \left(\frac{0}{50}\right)^2 - \left(\frac{0}{50}\right)^2 = 0$$

$$G_{1left} = 1 - \left(\frac{0}{100}\right)^2 - \left(\frac{50}{100}\right)^2 - \left(\frac{50}{100}\right)^2 = 0.5$$

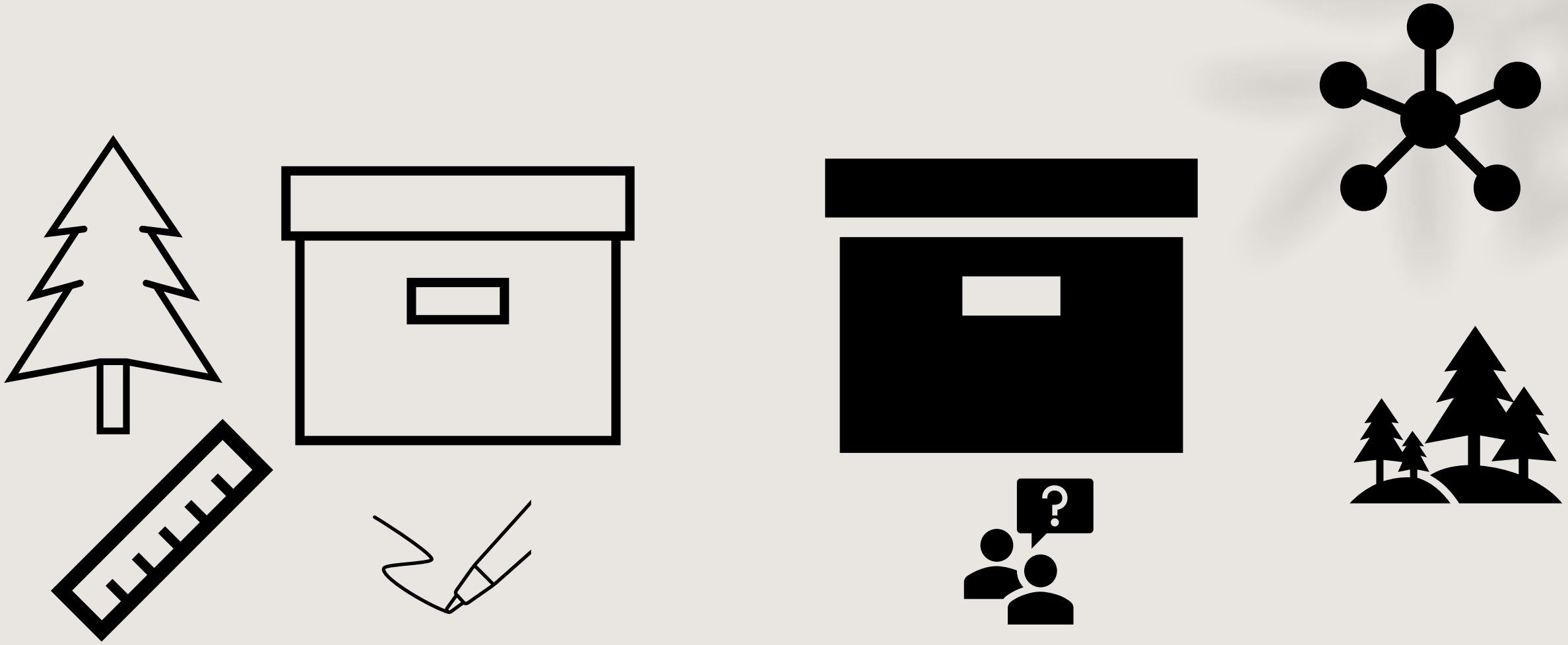
[0,49,5]
Versicolor

[0,1,45]
Virginica

$$G_{2left} = 1 - \left(\frac{0}{54}\right)^2 - \left(\frac{49}{54}\right)^2 - \left(\frac{5}{54}\right)^2 = 0{,}168$$

$$G_{2right} = 1 - \left(\frac{0}{46}\right)^2 - \left(\frac{1}{46}\right)^2 - \left(\frac{45}{46}\right)^2 = 0.043$$

INTERPRETACIÓN DEL MODELO: CAJA BLANCA FRENTE A CAJA NEGRA





Vamos a Python

Construcción del árbol

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import train_test_split

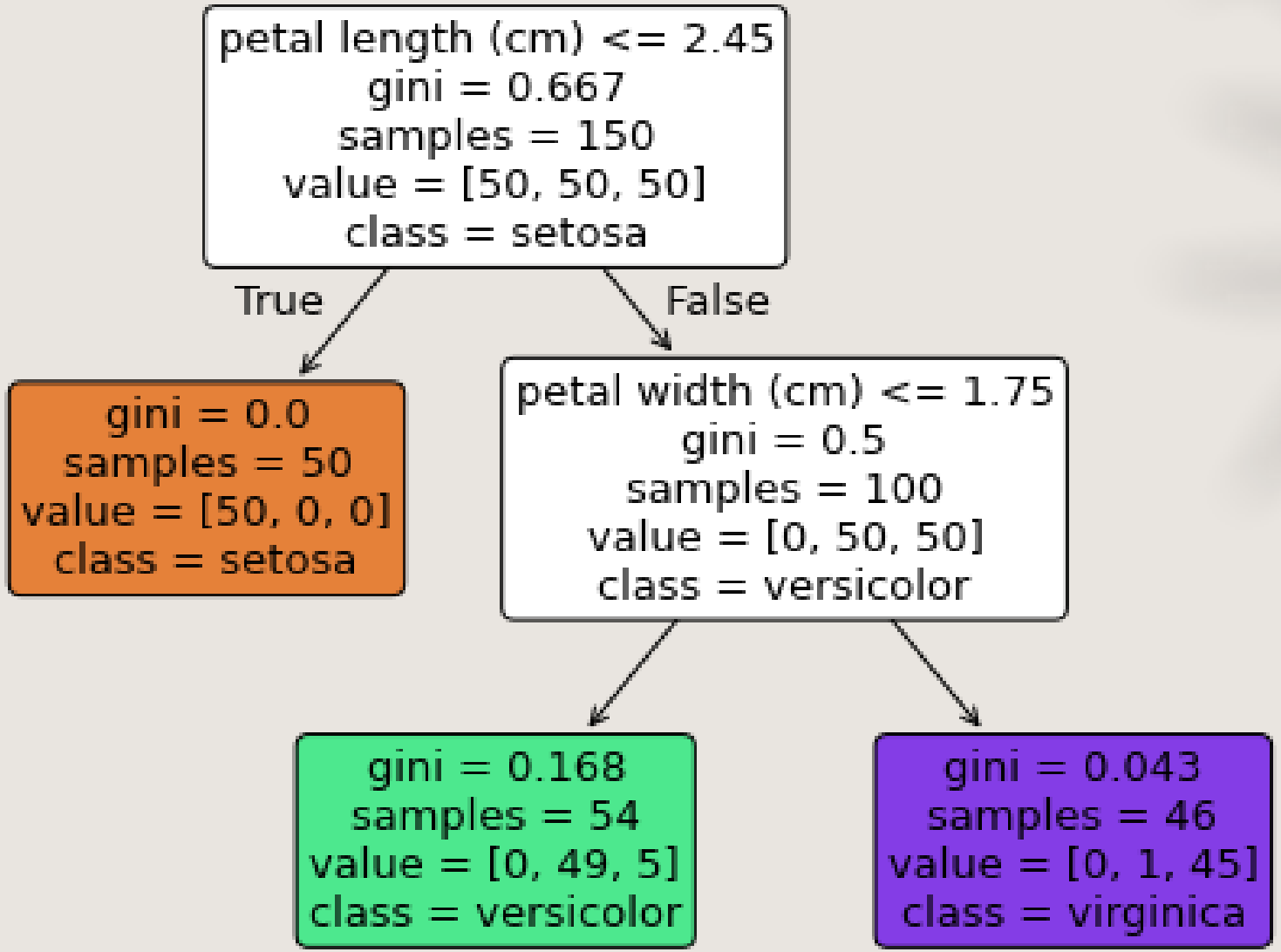

# Cargar el dataset Iris
iris = load_iris()


# Usar solo las dos características de pétalo
X = iris.data[:, 2:]  # petal length y petal width
y = iris.target


# Crear y entrenar el árbol de clasificación
tree_clf = DecisionTreeClassifier(max_depth=2, random_state=42)
tree_clf.fit(X, y)


# Visualizar el árbol
plt.figure(figsize=(8, 6))
plot_tree(tree_clf,
          feature_names=iris.feature_names[2:],  # Solo las dos variables
          class_names=iris.target_names,
          filled=True,
          rounded=True)
plt.title("Árbol de clasificación (Iris)")
plt.show()
```

Árbol de clasificación (Iris)



¿Qué información puede aparecer en un nodo?



- **Condición de división:** Ejemplo: petal length ≤ 2.45 regla que separa los datos en ese nodo.
- **Número de muestras (samples)**: Cantidad de observaciones que llegaron a ese nodo.
- **Distribución de clases (value)**

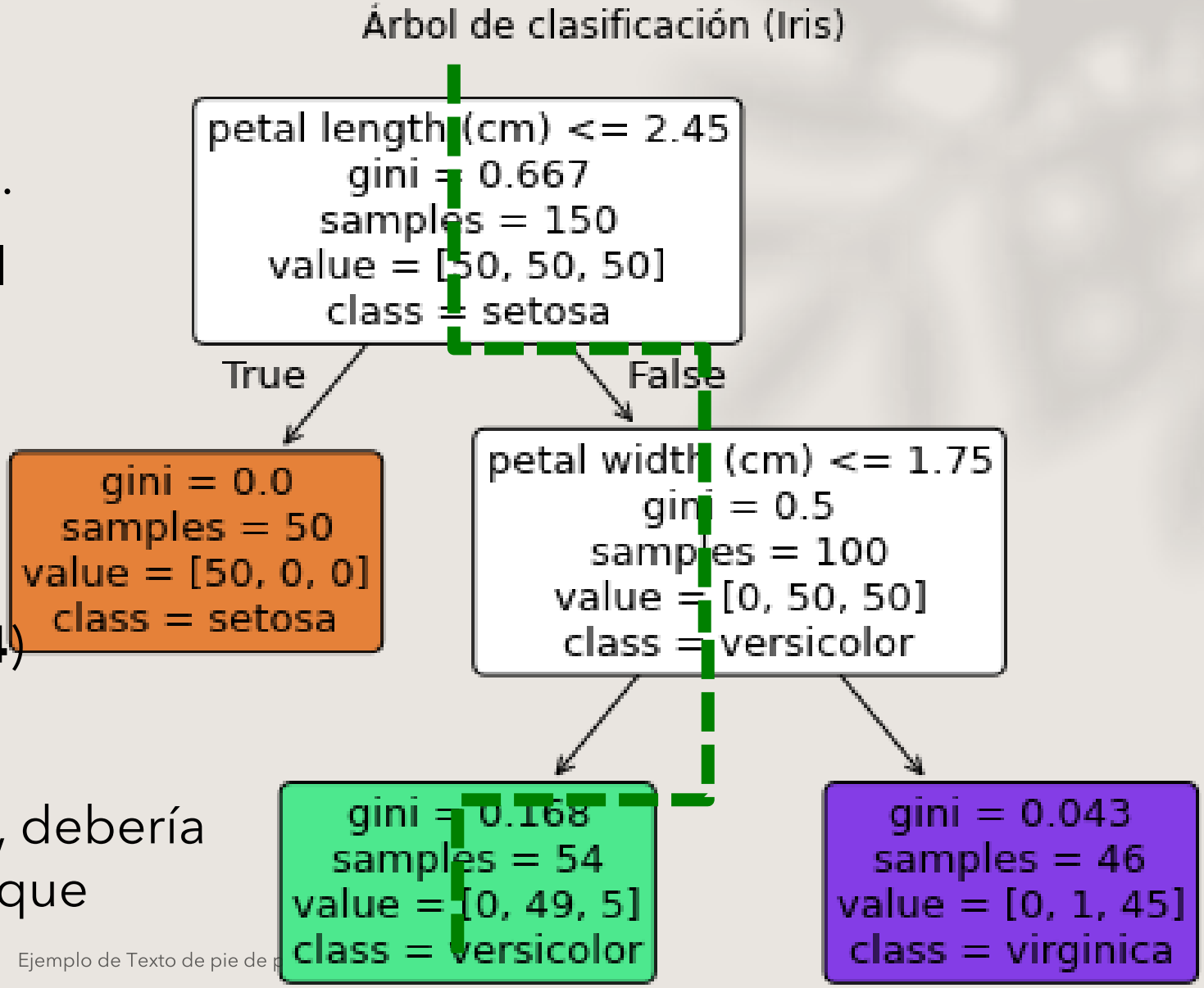
  Ejemplo: [50, 0, 0] para tres clases (Setosa, Versicolor, Virginica). Muestra cuántos elementos de cada clase hay en el nodo.
- **Clase predicha (class):** La clase mayoritaria en ese nodo.
- **Impureza (Gini o Entropía):** Valor que indica qué tan mezcladas están las clases en el nodo.

Estimación de probabilidades de clase

Se ha encontrado una flor cuyos pétalos miden:
5 cm de largo y **1,5 cm de ancho**.

El nodo hoja correspondiente es el nodo situado a la izquierda de la profundidad 2, por lo que el árbol de decisión debería generar las siguientes probabilidades:
0 % para Iris **setosa (0/54),**
90,7 % para **Iris versicolor** (**49/54**)
y **9,3 %** para **Iris virginica** (**5/54**).

Y si se le pide que prediga la clase, debería generar Iris versicolor (clase 1) porque tiene la mayor probabilidad.



Estimación de probabilidades de clase

Se ha encontrado una flor cuyos pétalos miden:

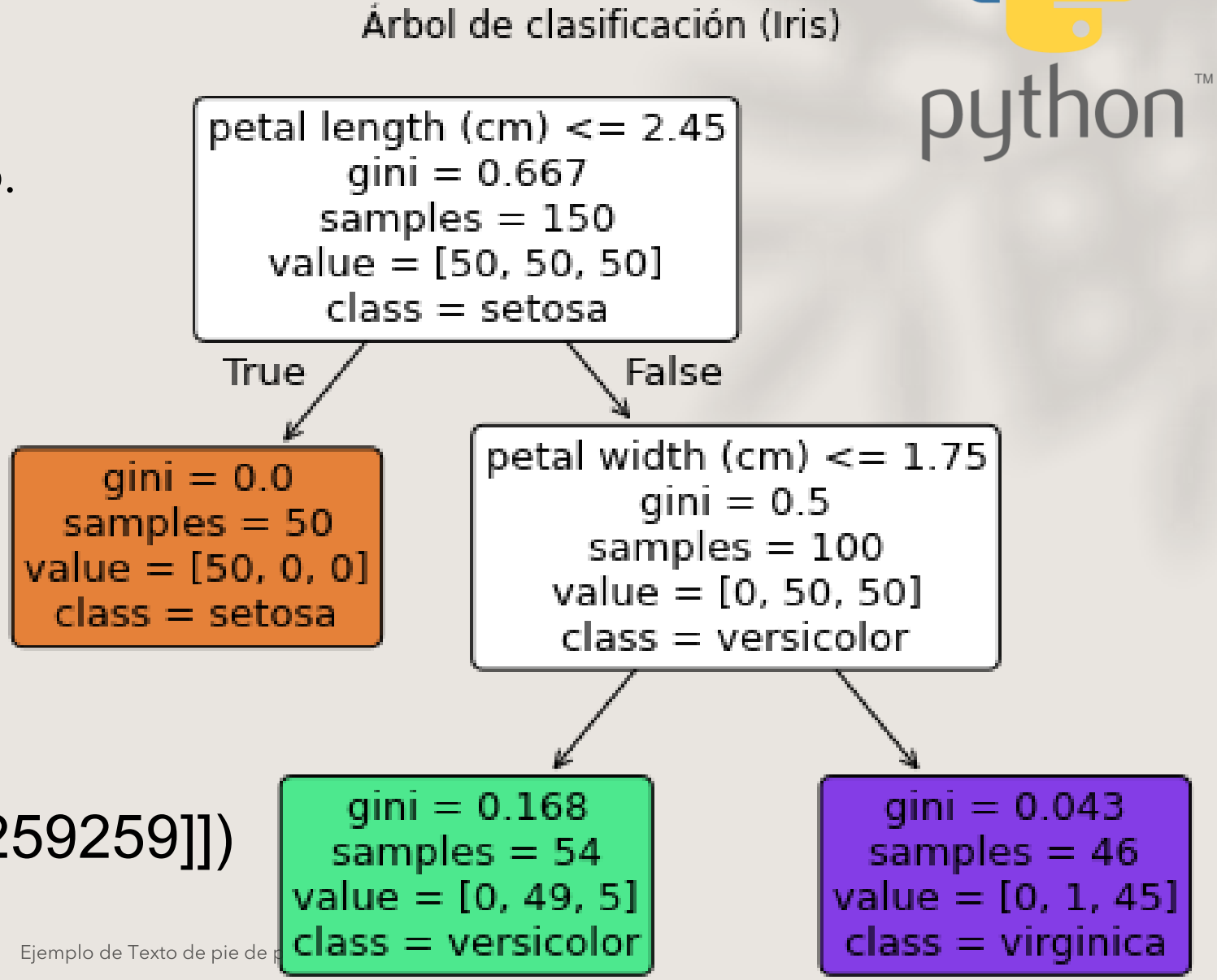
5 cm de largo y **1,5 cm de ancho**.

tree_clf.predict_proba([[5, 1.5]])

Calcula las **probabilidades** de que la muestra [petal length = 5, petal width = 1.5] pertenezca a cada clase.

Resultado:

array([[0. , 0.90740741, 0.09259259]])



array([[0. , 0.90740741, 0.09259259]])

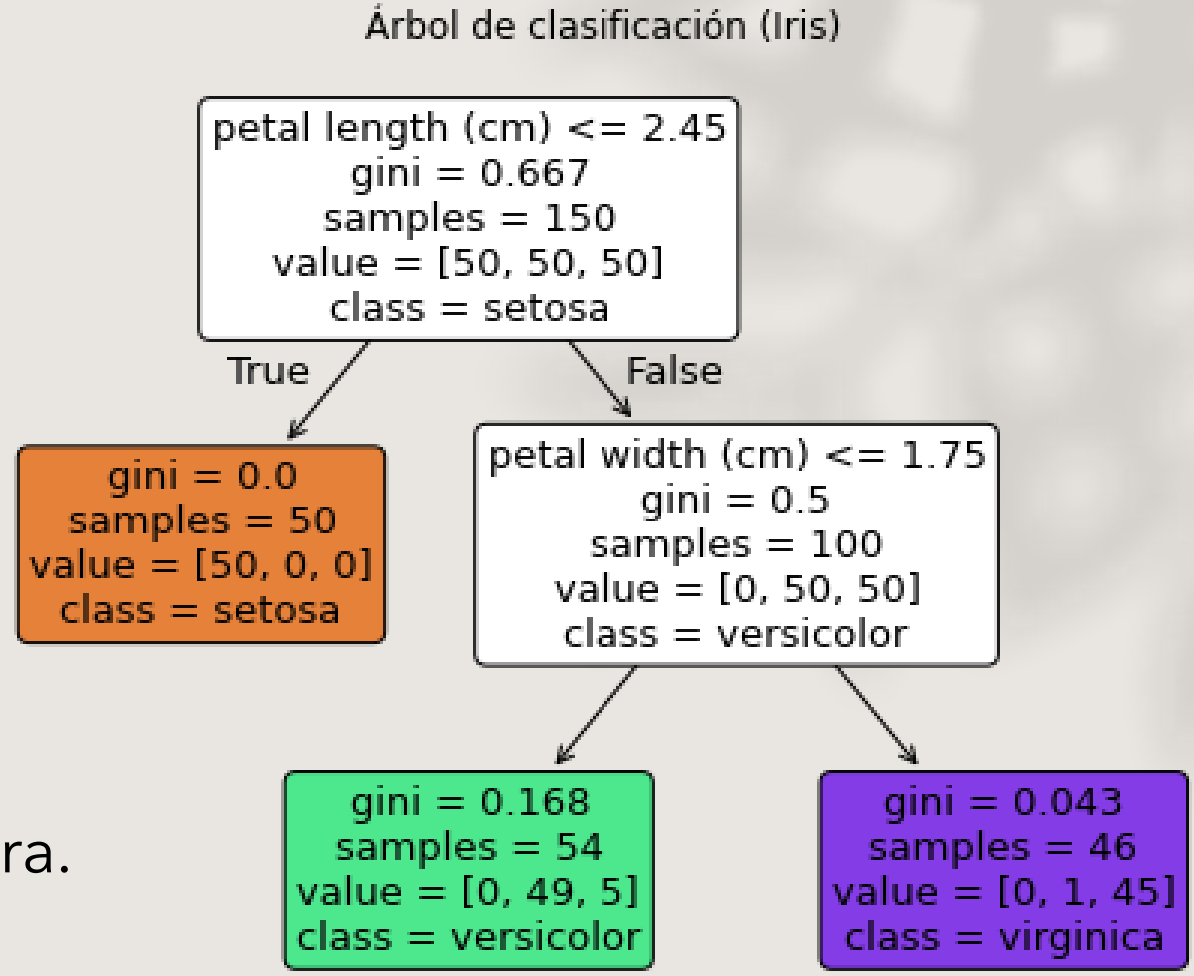
Significa:

Clase 0 (*setosa*): 0.0 (0%)

Clase 1 (*versicolor*): 0.9074 (≈ 90.7%)

Clase 2 (*virginica*): 0.0926 (≈ 9.3%)

Esto se calcula según la proporción de muestras en la hoja del árbol donde cae el punto.



tree_clf.predict([[5, 1.5]])

Devuelve la **clase predicha** para esa muestra.

Resultado:

array([1])

Significa que el modelo predice la clase **1** → **versicolor**, porque es la clase con mayor probabilidad.

```
[44]
1 tree_clf.predict_proba([[5, 1.5]])  #Calcula las probabilidades de que la muestra [petal length = 5, petal width = 1.5] pertenezca a cada clase.
```
Python

```
array([[0.        , 0.90740741, 0.09259259]])
```

```
[47]
1 tree_clf.predict([[5, 1.5]])
```
Python

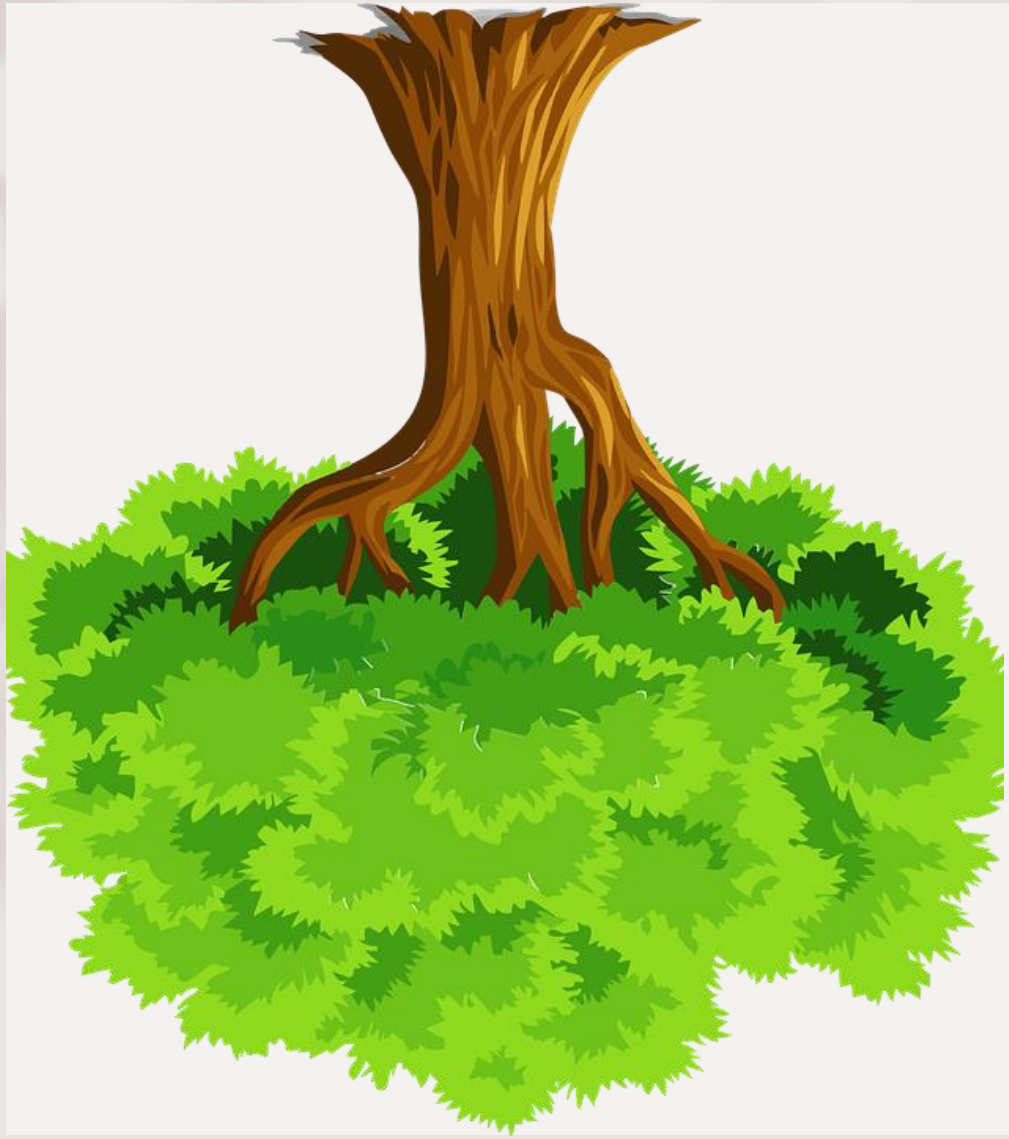
```
array([1])
```

```
[49]
1 tree_clf.predict_proba([[6, 2.5]])  #Calcula las probabilidades de que la muestra [petal length = 5, petal width = 1.5] pertenezca a cada clase.
```
Python

```
array([[0.        , 0.02173913, 0.97826087]])
```

```
[50]
1 tree_clf.predict([[5, 2.5]])
```
Python

```
array([2])
```



¿Cómo se construye este árbol de forma automática?

¿Cómo se logra que se construyan las agrupaciones lo más homogeneas posibles?

El algoritmo de entrenamiento CART

- El algoritmo funciona dividiendo primero el conjunto de entrenamiento en dos subconjuntos mediante una **característica k y un umbral t** (por ejemplo:

  "longitud del pétalo ≤ 2,45 cm").
- ¿Cómo elige k y t? Busca el par (k, t) que produce los subconjuntos más puros (ponderados por su tamaño).
- **Minimiza** la siguiente función de costo

$$J(k, t_k) = \frac{m_{left}}{m} G_{left} + \frac{m_{right}}{m} G_{right}$$

Donde

$G_{\frac{left}{right}}$ mide la impureza del subconjunto izquierdo/derecho

$m_{left/right}$ el número de instancias en el subconjunto izquierdo/derecho.

CART es un *greedy algorithm* (algoritmo voraz)

Definición

- Elije en cada paso la opción que parece mejor en ese momento, sin considerar el impacto futuro.
- Es decir, **toma decisiones localmente óptimas** esperando que lleven a una solución globalmente óptima.

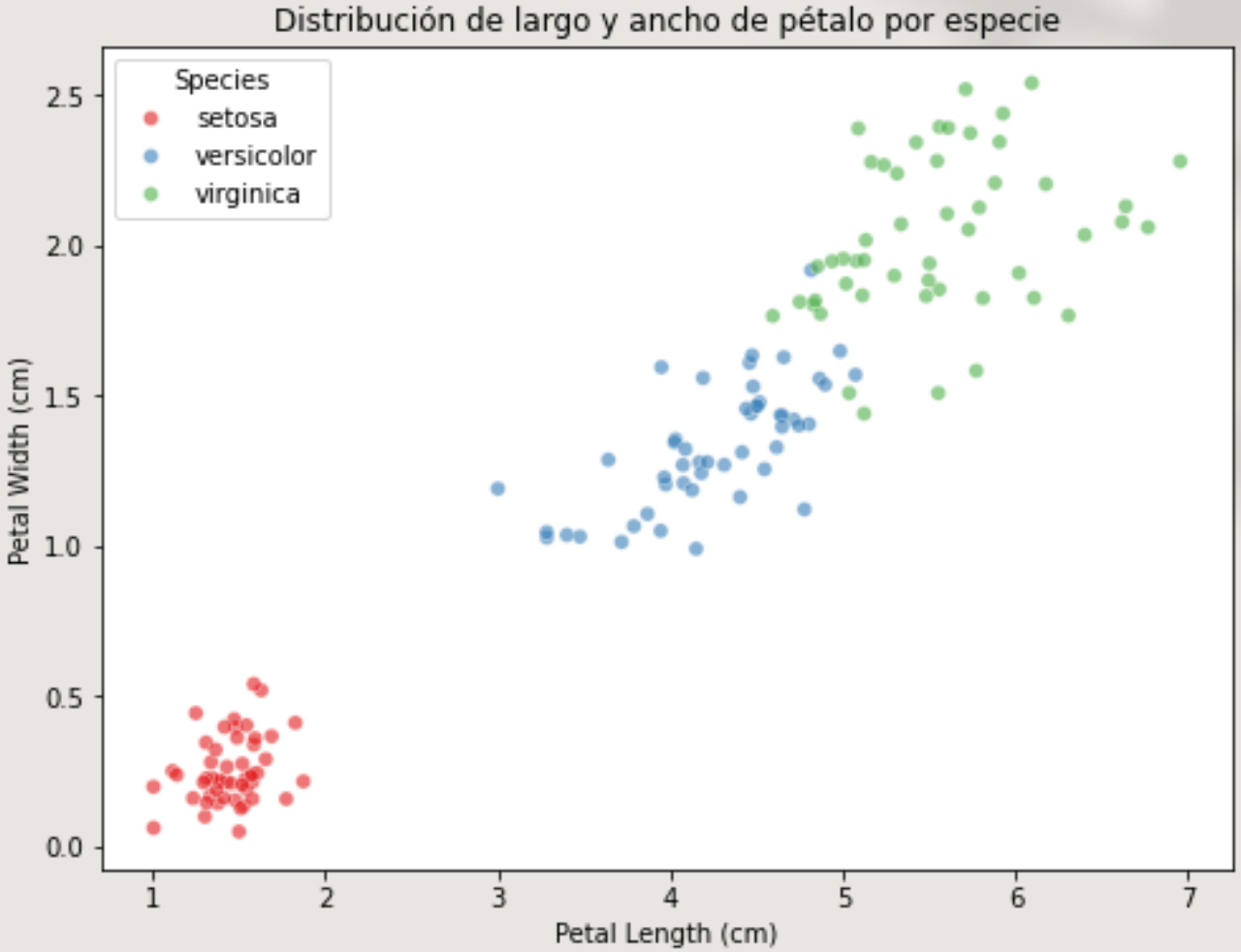


Características

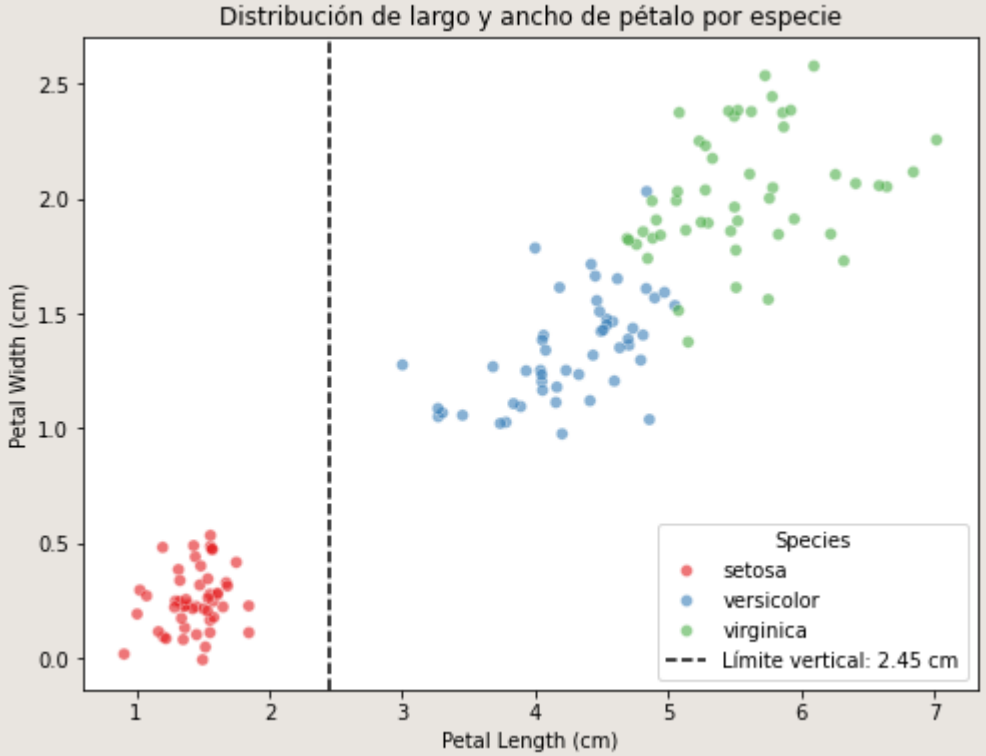
- **Simple y rápido**: porque no explora todas las posibilidades.
- **No siempre garantiza la mejor solución global**, pero funciona bien en problemas donde la elección local óptima conduce a la solución óptima global (propiedad de optimalidad).

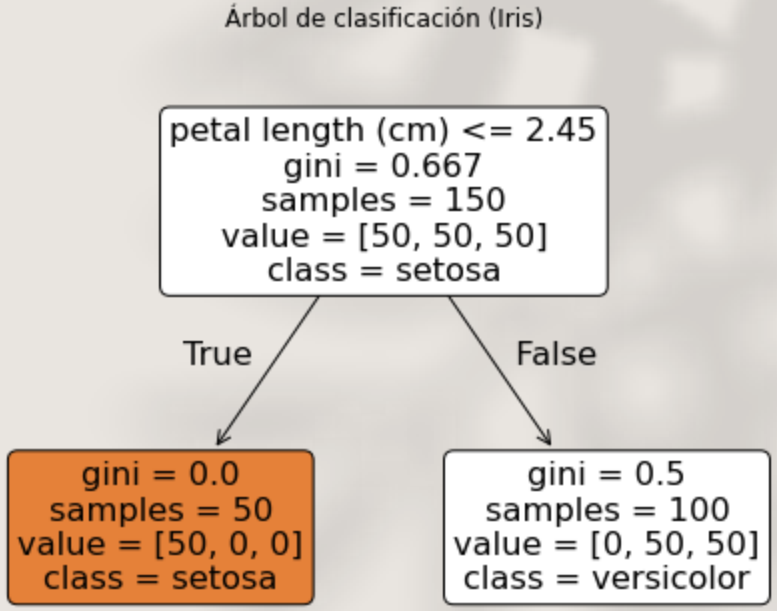
Se acuerdan cuando teniamos este grafico

Calculemos la función de costo al pasar de la profundidda 0 a la 1.



Partiendo de pethal length

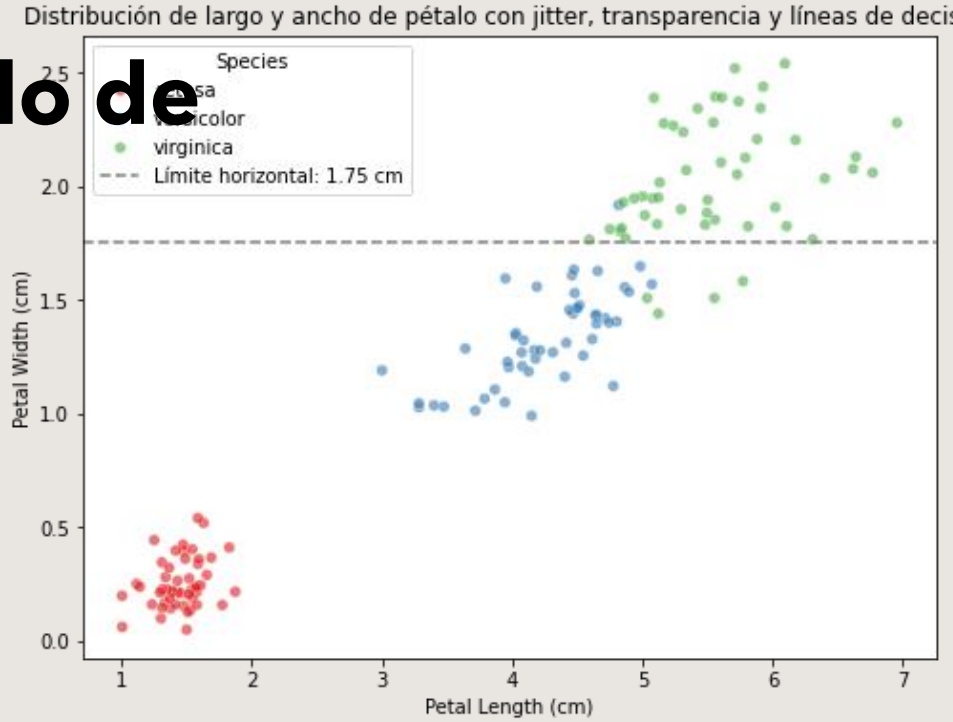




$$J(k, t_k) = \frac{m_{left}}{m} G_{left} + \frac{m_{right}}{m} G_{right}$$

$$J(k = 1, t_1 = 2.45) = \frac{(50 + 0 + 0)}{150} \times 0 + \frac{(0 + 50 + 50)}{150} \times 0.5 = 0{,}333$$

Partiendo de pethal width



Petal width (cm) $\leq$ 1,75
Gini=0.667
Samples=150
Values=[50,50,50]

Gini=0.5446
Samples=104
Values=[50,49,5]

Gini=0.0425
Samples=46
Values=[0,1,45]

$$G_i = 1 - \sum_{k=1}^{K} p_{i,k}^2$$

$$G_{1left} = 1 - \left(\frac{50}{104}\right)^2 - \left(\frac{49}{104}\right)^2 - \left(\frac{5}{104}\right)^2 = 0{,}5446$$

$$G_{1right} = 1 - \left(\frac{0}{46}\right)^2 - \left(\frac{1}{46}\right)^2 - \left(\frac{45}{46}\right)^2 = 0{,}0425$$

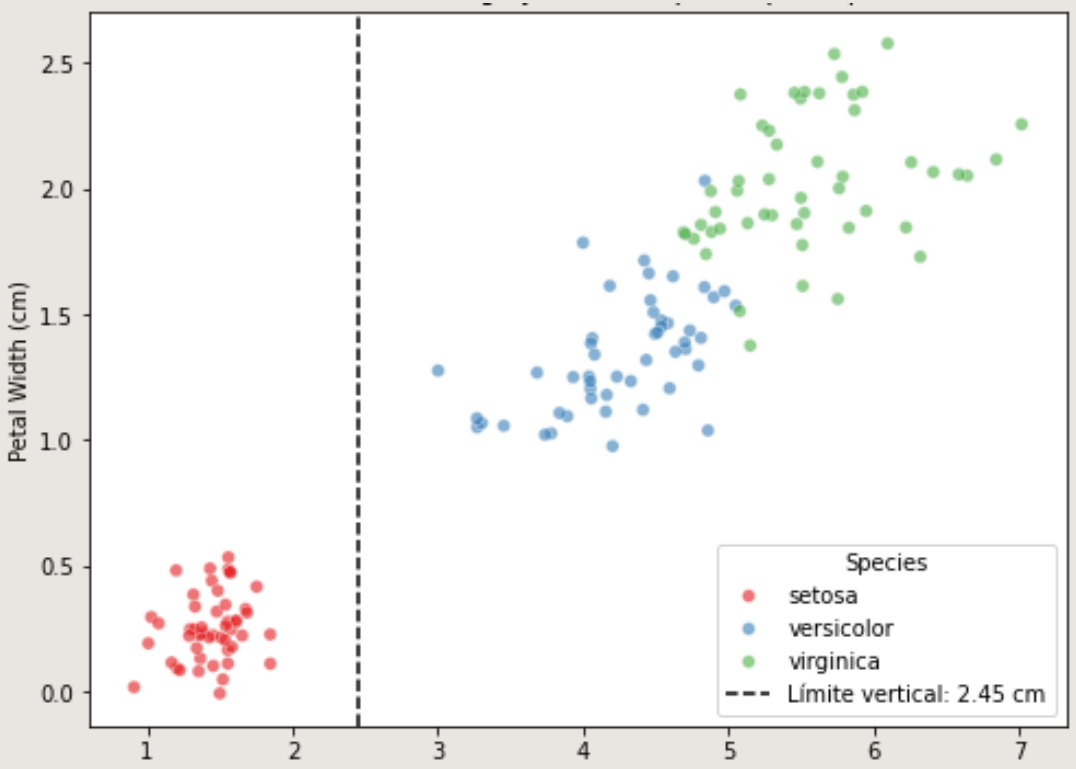
$$J(k, t_k) = \frac{m_{left}}{m} G_{left} + \frac{m_{right}}{m} G_{right}$$

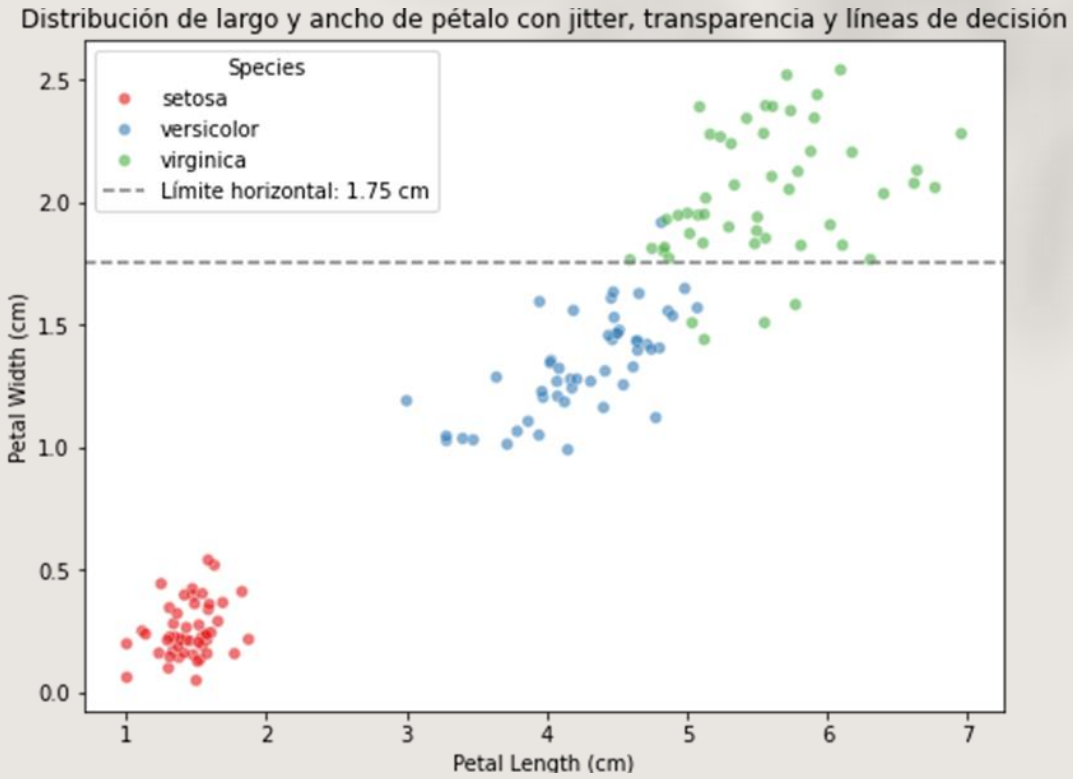
$$J(k = 1, t_1 = 1.75) = \frac{(50 + 49 + 5)}{150} \times 0{,}5446 + \frac{(0 + 1 + 45)}{150} \times 0{,}0425 = 0{,}3906$$

$J(k = 1, t_1 = 2.45) = 0.333$

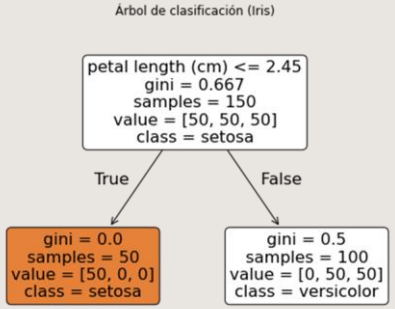
$<$

$J(k = 1, t_1 = 1.75) = 0{,}3906$





Algoritmo CART

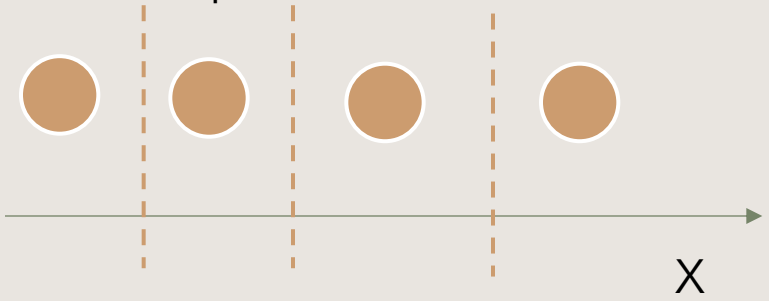


Mínimo $J(k, t_k)$

Que los nodos sean puros (**Gini = 0**), O se **alcance un criterio de parada** (profundidad máxima, mínimo de muestras, etc.).

Calcular todos los posibles umbrales → **Calcular índices Gini y costo** → **Escoger el umbral con menor costo** → **Repetir**

Son n-1 para cada variable



$$G_i = 1 - \sum_{k=1}^{K} p_{i,k}^2$$

$$J(k, t_k) = \frac{m_{left}}{m} G_{left} + \frac{m_{right}}{m} G_{right}$$

Complejidad Computacional

Para hacer una predicción, el árbol se recorre desde la raíz hasta una hoja.

Como los árboles suelen estar balanceados, este recorrido implica pasar por aproximadamente **O(log(m))** nodos, siendo m el número de muestras.

En cada nodo solo se evalúa una característica, por lo que la predicción es rápida y no depende del número de características



¿Qué significa O(log(m))?

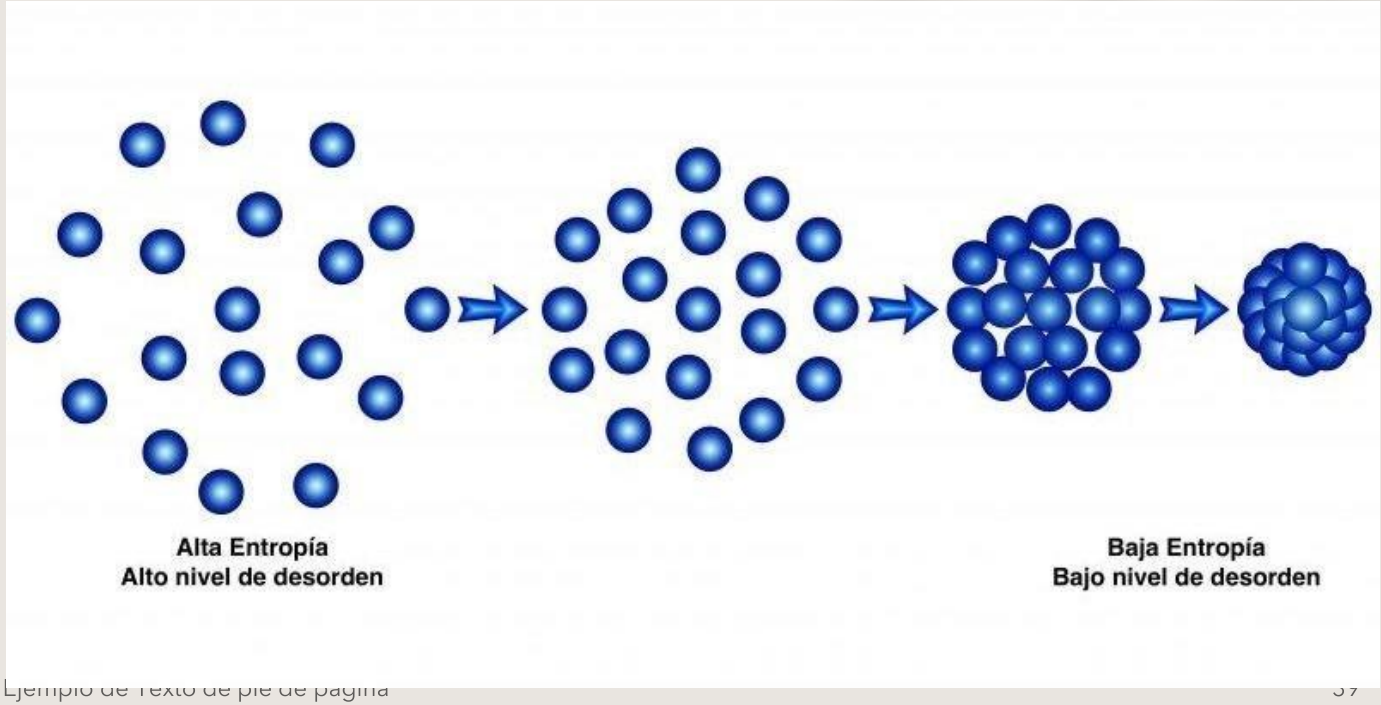
- "**O**" es la notación de "orden de magnitud" o "Big O", que indica el comportamiento del algoritmo en el peor caso.
- "**log(m)**" significa que el tiempo de ejecución crece de forma logarítmica con respecto al número de muestras m.
- 🔍 Ejemplo intuitivo:
- Si tenés:**1.000 muestras** → el algoritmo recorre aproximadamente $\log_2(1000) \approx$ **10 nodos**.
- **1.000.000 muestras** → recorre $\log_2(1.000.000) \approx$ **20 nodos**.
- Aunque el número de muestras crezca mucho, el número de pasos necesarios para hacer una predicción crece muy lentamente.



¿Impureza de Gini o entropía?

- Por defecto, se usa **impureza de Gini** para dividir nodos.
- Se puede usar **entropía** configurando el hiperparámetro criterion="entropy".

$$H_i = -\sum_{\substack{k=1 \\ p_{i,k} \neq 0}}^{n} p_{i,k} \log_2 (p_{i,k})$$

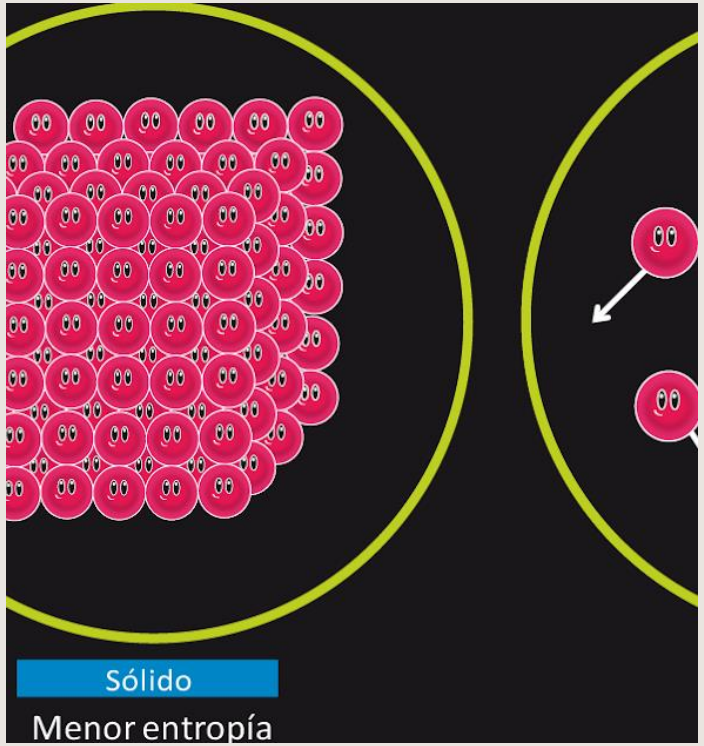


Origen y significado de la entropía

En teoría de la información, mide el contenido informativo promedio de un mensaje.



Proviene de la termodinámica como medida de desorden



En aprendizaje automático, mide la impureza: entropía = 0 cuando todas las instancias son de una sola clase



Diferencias prácticas entre Gini y entropía

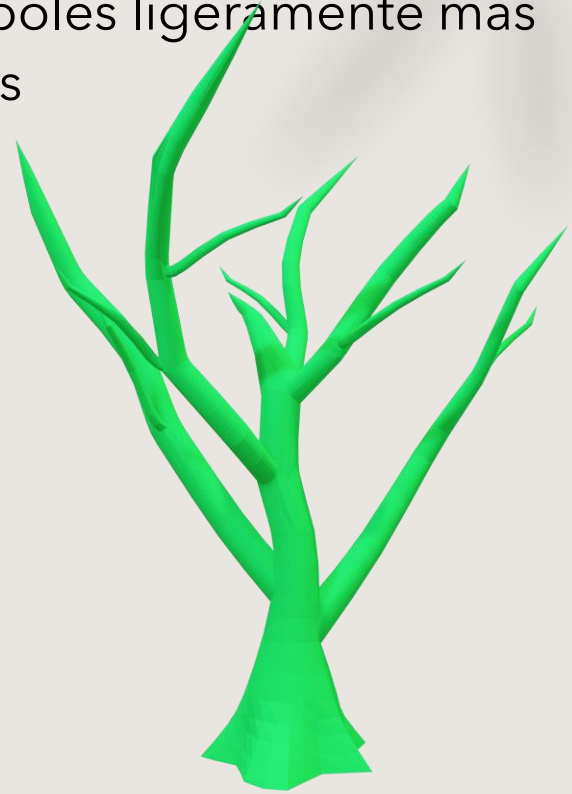
GINI

- se calcula más rápido → opción por defecto
- tiende a aislar la clase más frecuente en una rama.



ENTROPIA

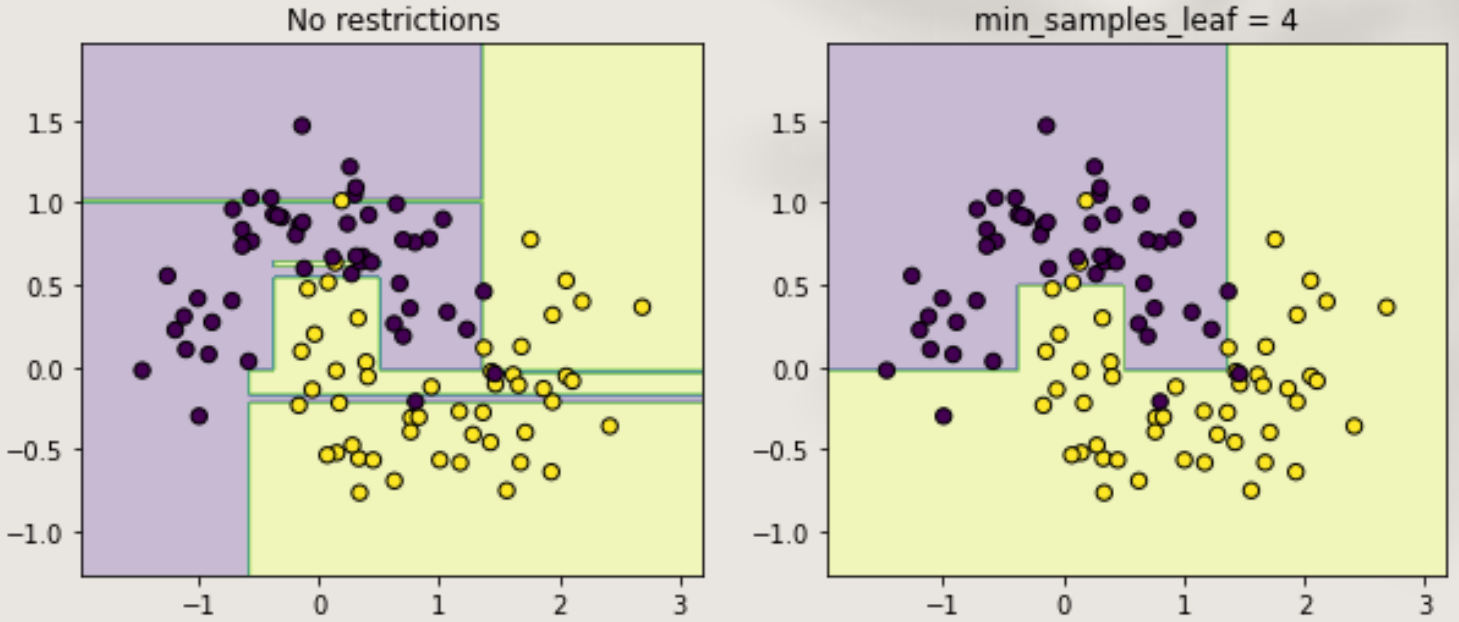
- produce árboles ligeramente más equilibrados





Naturaleza del modelo

- Los árboles de decisión son **modelos no paramétricos**: no tienen un número fijo de parámetros antes del entrenamiento, su estructura se adapta a los datos.
- Esto les da mucha flexibilidad, pero también **alto riesgo de sobreajuste** si no se aplican restricciones.



Se definen zonas muy pequeñas, el set de datos se ajusta muy bien a los datos de entrenamiento pero no predice correctamente



Prevenir el sobreajuste



Restringir el número de nodos

Pre poda

Definir el valor de los hiperparámetros:

- Profundidad máxima
- Número mínimo de datos en un nodo
- Número mínimo de datos en una hoja



Post poda

Poda de complejidad de costos, paramétro alfa. **¿Para qué sirve?**
Si α = 0 → no se penaliza la complejidad → árbol muy grande (riesgo de sobreajuste).
Si α aumenta → se penaliza la complejidad → el árbol se poda más → modelo más simple (menos sobreajuste, pero puede perder precisión).

Árboles de clasificación

Ventajas

- Fácil de interpretar
- Se puede ver cuáles son las características más relevantes

Desventajas

- El tiempo de entrenamiento crece con el numero de datos y características.





Observando el árbol

Las variables que aparecen **más cerca de la raíz** (arriba) son las más importantes porque se usan primero para dividir los datos.

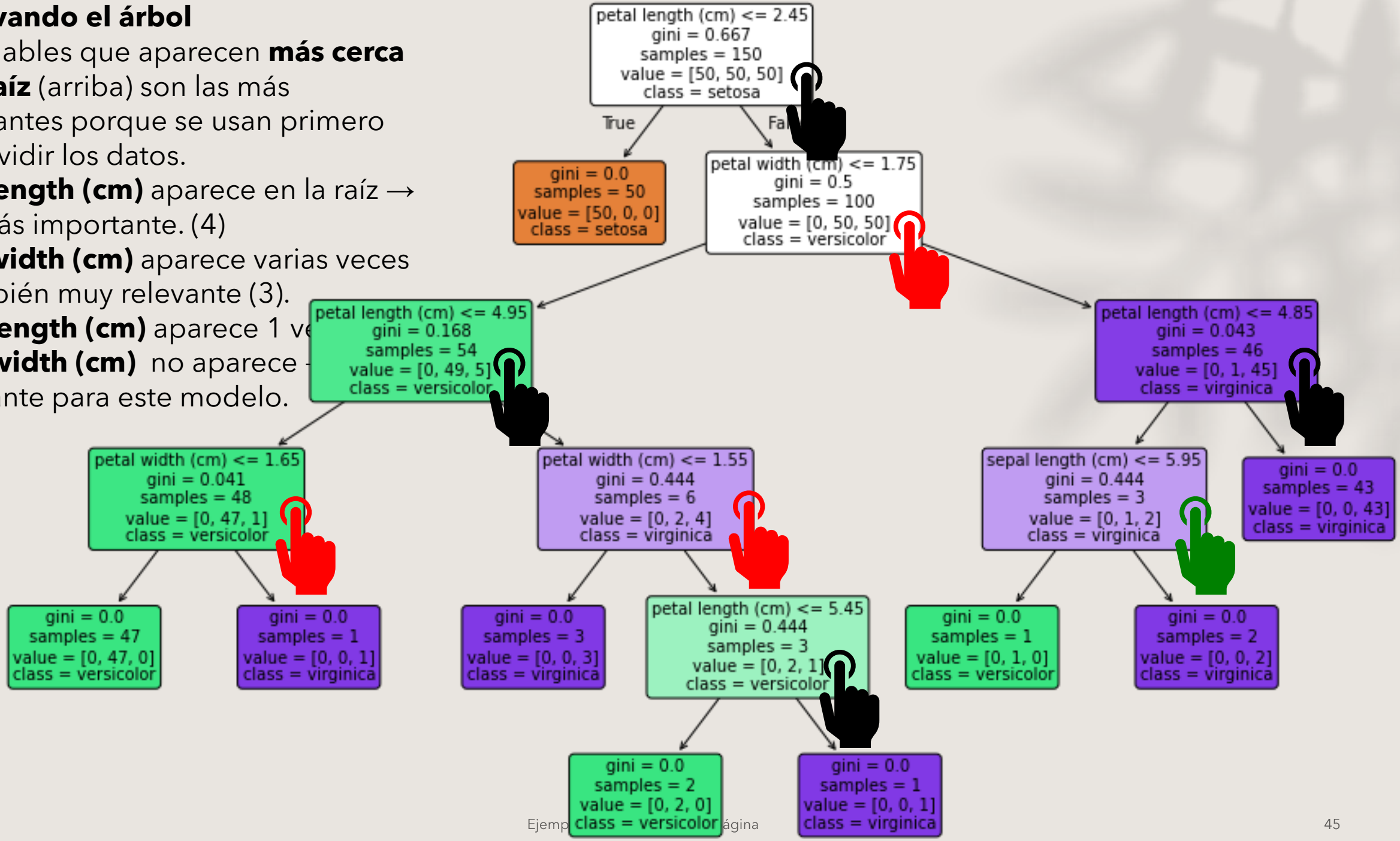
petal length (cm) aparece en la raíz → es la más importante. (4)

petal width (cm) aparece varias veces → también muy relevante (3).

sepal length (cm) aparece 1 ve

sepal width (cm) no aparece

irrelevante para este modelo.



```python
import pandas as pd

# Importancia de las variables
importances = tree_clf.feature_importances_
features = iris.feature_names

# Mostrar en tabla ordenada
pd.DataFrame({'Feature': features, 'Importance': importances}).sort_values(by='Importance', ascending=False)
```

[5] ✓ 0.7s

	Feature	Importance
2	petal length (cm)	0.893264
3	petal width (cm)	0.087626
1	sepal width (cm)	0.019110
0	sepal length (cm)	0.000000

Árboles de regresión



Esta foto de Autor desconocido está bajo licencia CC BY

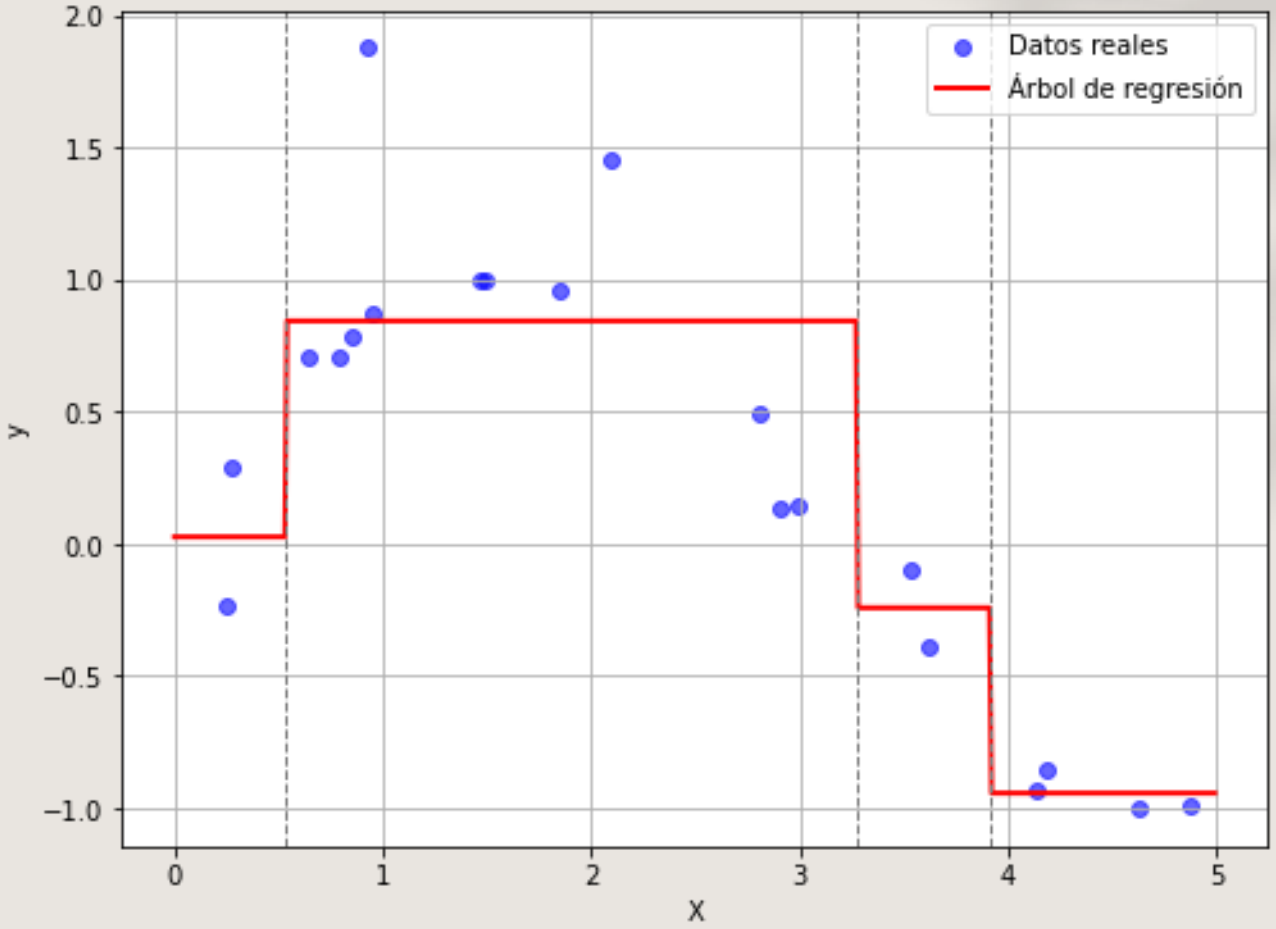
Por qué se ve así

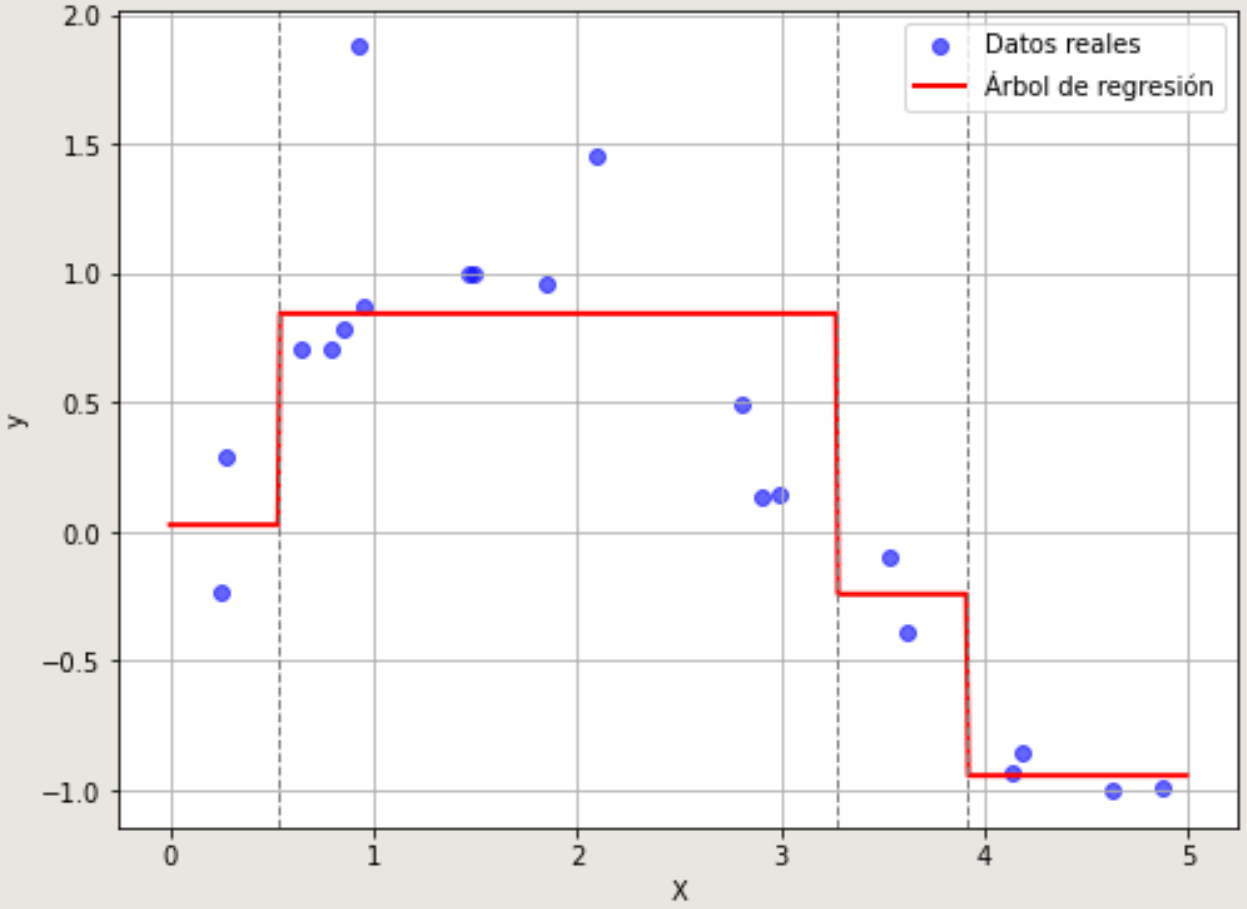
Un árbol de regresión **divide el espacio en intervalos** (segmentos) según los umbrales que aprendió.

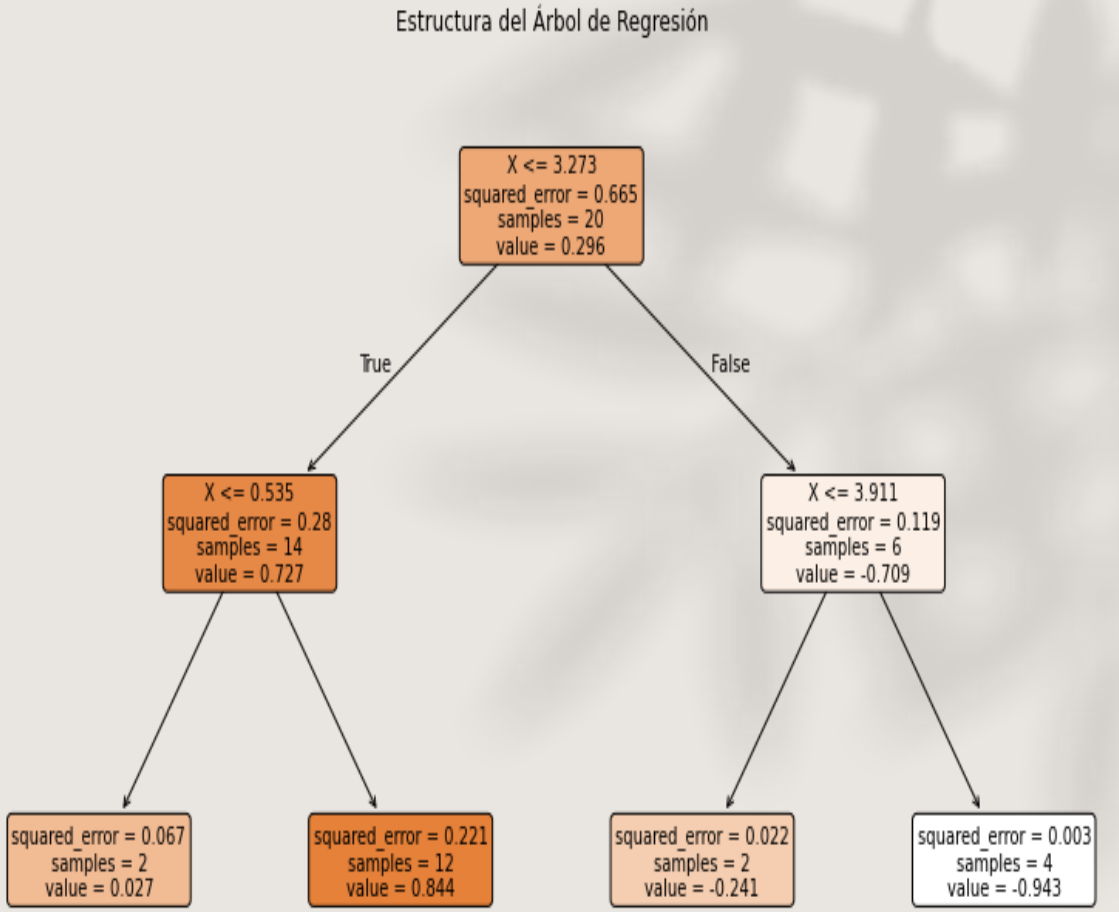
Dentro de cada intervalo, el árbol **predice el promedio** de los valores de y en ese segmento.

Por eso la línea roja no es suave como una regresión lineal, sino que tiene **tramos horizontales**:

- Cada tramo corresponde a una **hoja del árbol**.
- Cuando el árbol cambia de hoja, la línea salta a otro nivel.







Cómo interpretarlo

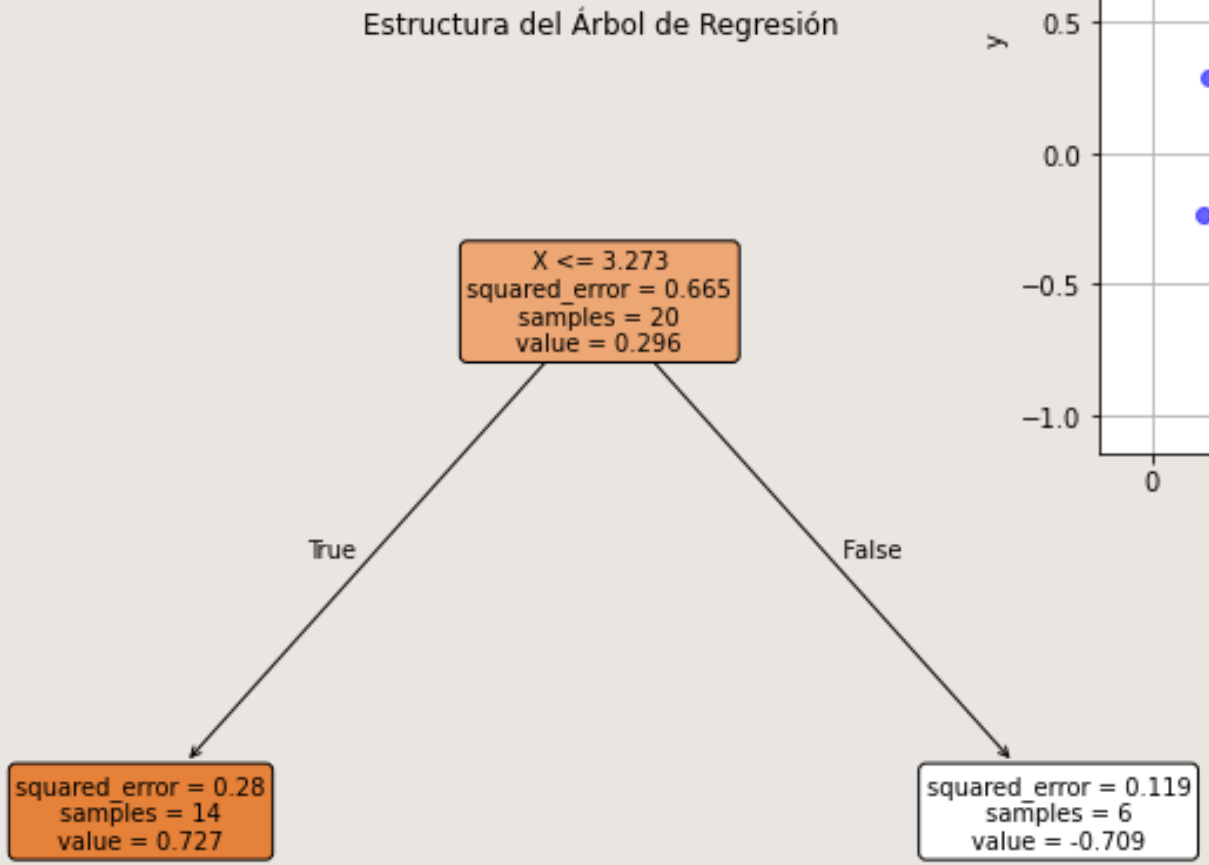
Las **líneas grises punteadas** son los **puntos de división** (thresholds) del árbol.

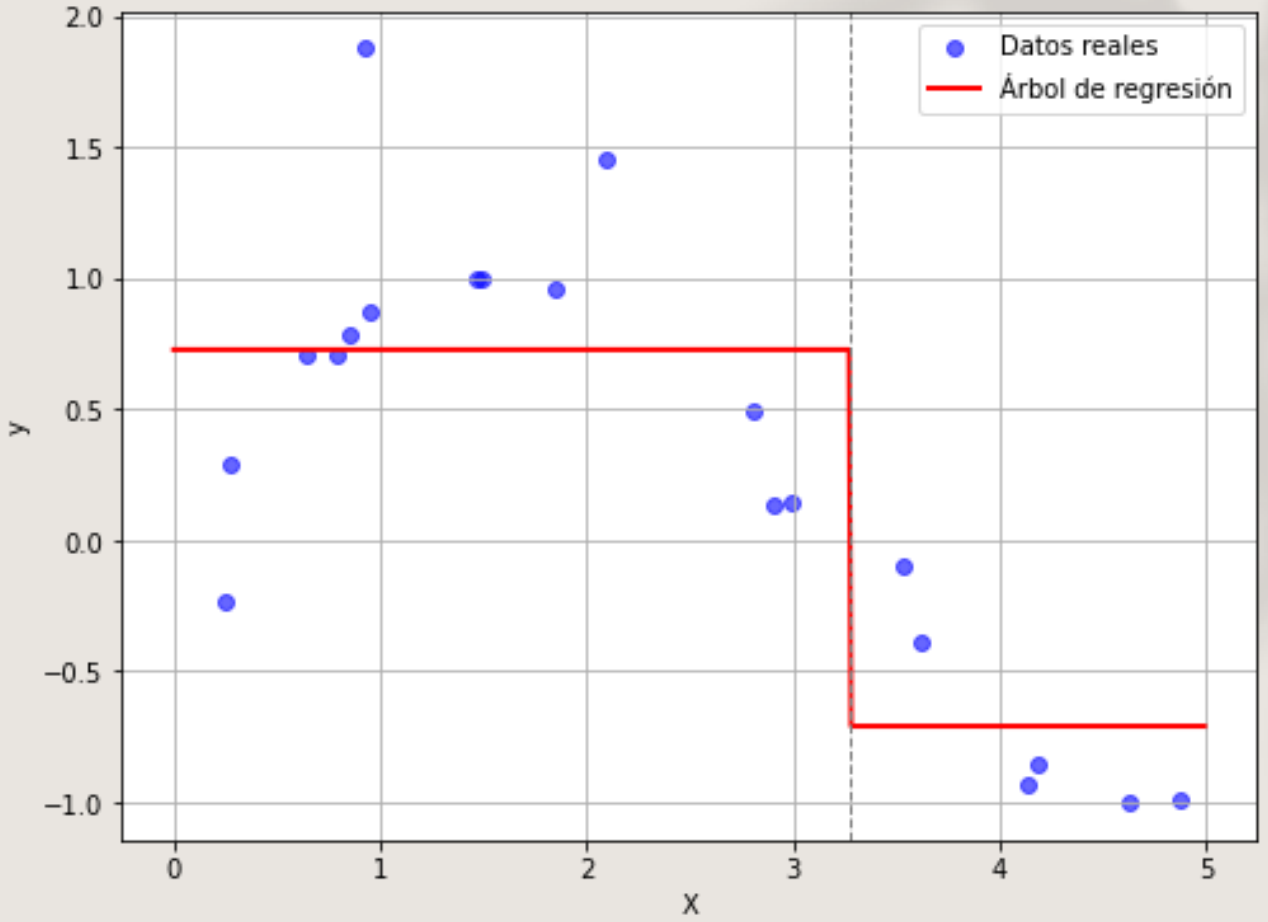
Entre dos líneas grises, la predicción es constante (el promedio de los datos en esa hoja).

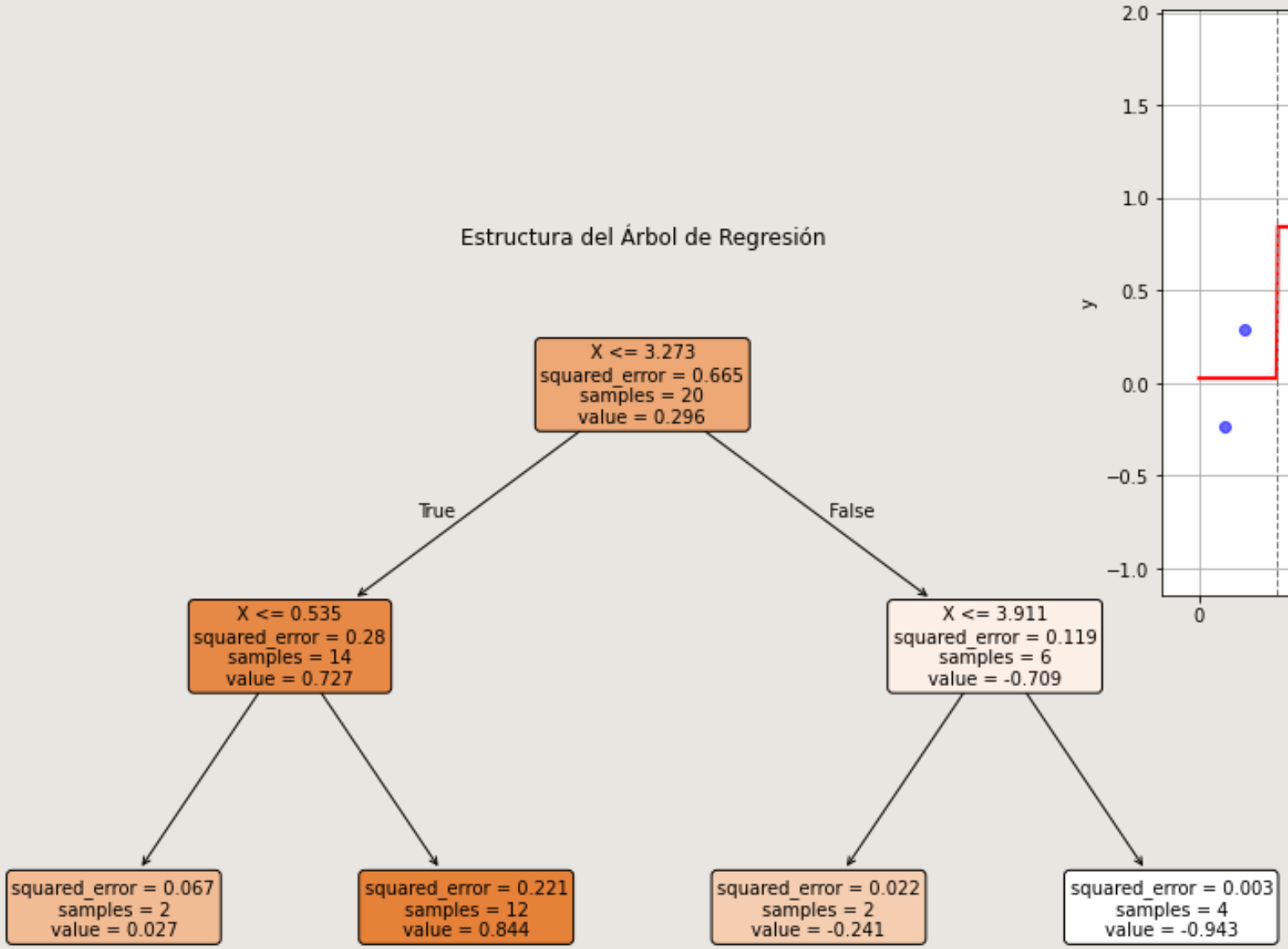
Si se aumentá max_depth, el árbol tendrá más divisiones → más tramos → se ajusta más a los datos (sobreajuste).

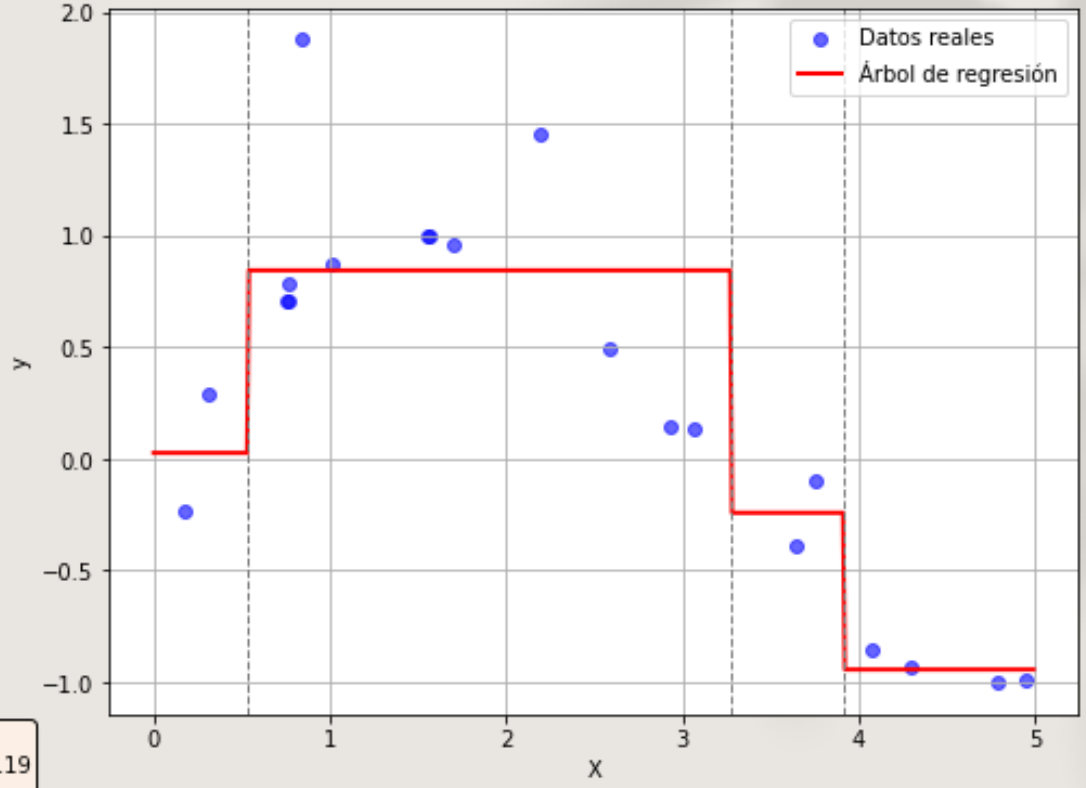
Si se reduce max_depth, el árbol tendrá menos divisiones → menos tramos → modelo más simple (subajuste).

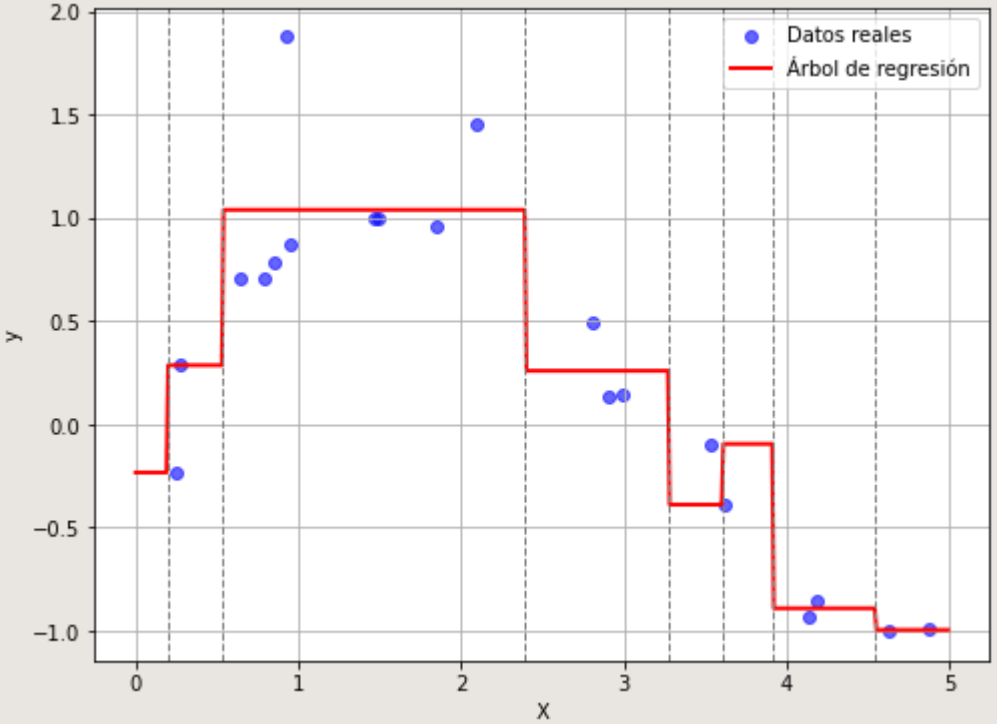
Paso a paso

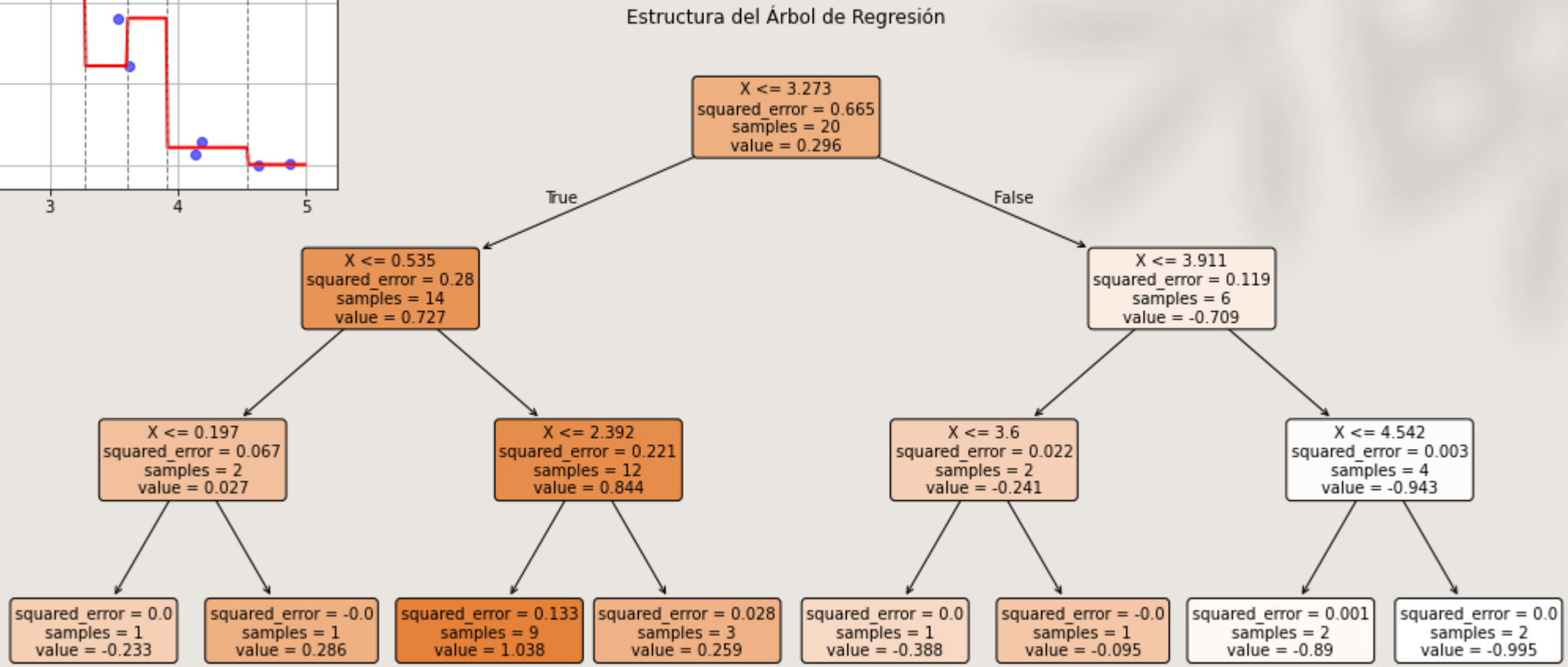

Estructura del Árbol de Regresión
X <= 3.273
squared_error = 0.665
samples = 20
value = 0.296
True
False
squared_error = 0.28
samples = 14
value = 0.727
squared_error = 0.119
samples = 6
value = -0.709


Datos reales
Árbol de regresión
y
X
2.0
1.5
1.0
0.5
0.0
−0.5
−1.0
0
1
2
3
4
5


Estructura del Árbol de Regresión
X <= 3.273
squared_error = 0.665
samples = 20
value = 0.296
True
False
X <= 0.535
squared_error = 0.28
samples = 14
value = 0.727
X <= 3.911
squared_error = 0.119
samples = 6
value = -0.709
squared_error = 0.067
samples = 2
value = 0.027
squared_error = 0.221
samples = 12
value = 0.844
squared_error = 0.022
samples = 2
value = -0.241
squared_error = 0.003
samples = 4
value = -0.943

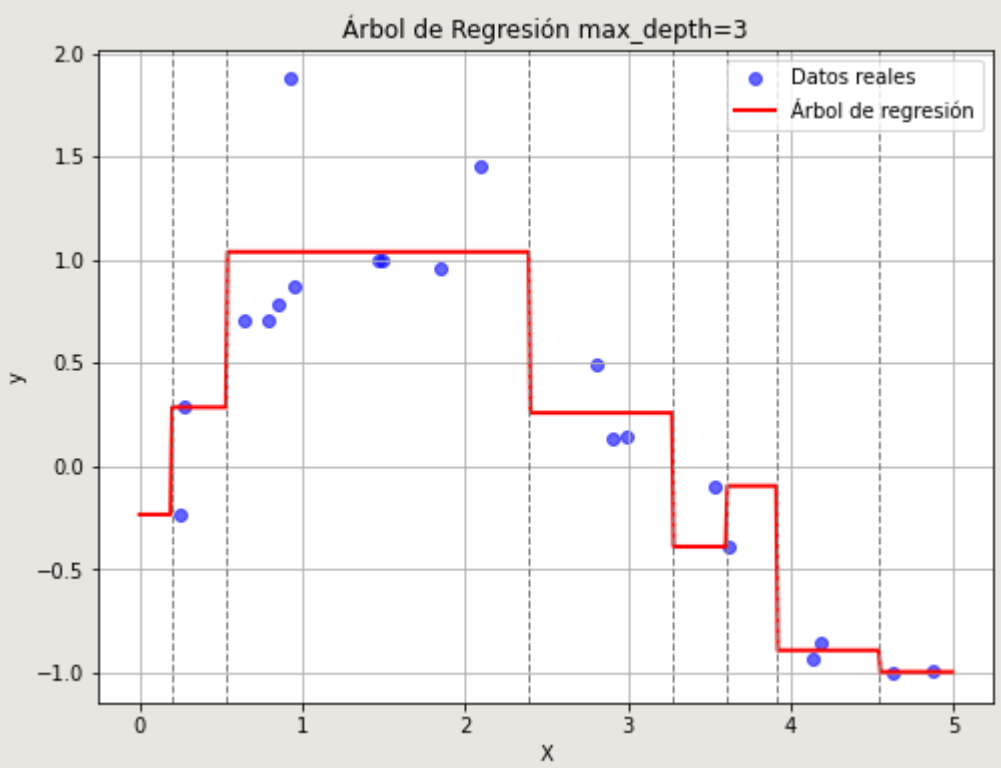

Datos reales
Árbol de regresión
y
X
2.0
1.5
1.0
0.5
0.0
−0.5
−1.0
0
1
2
3
4
5


Datos reales
Árbol de regresión
X
y

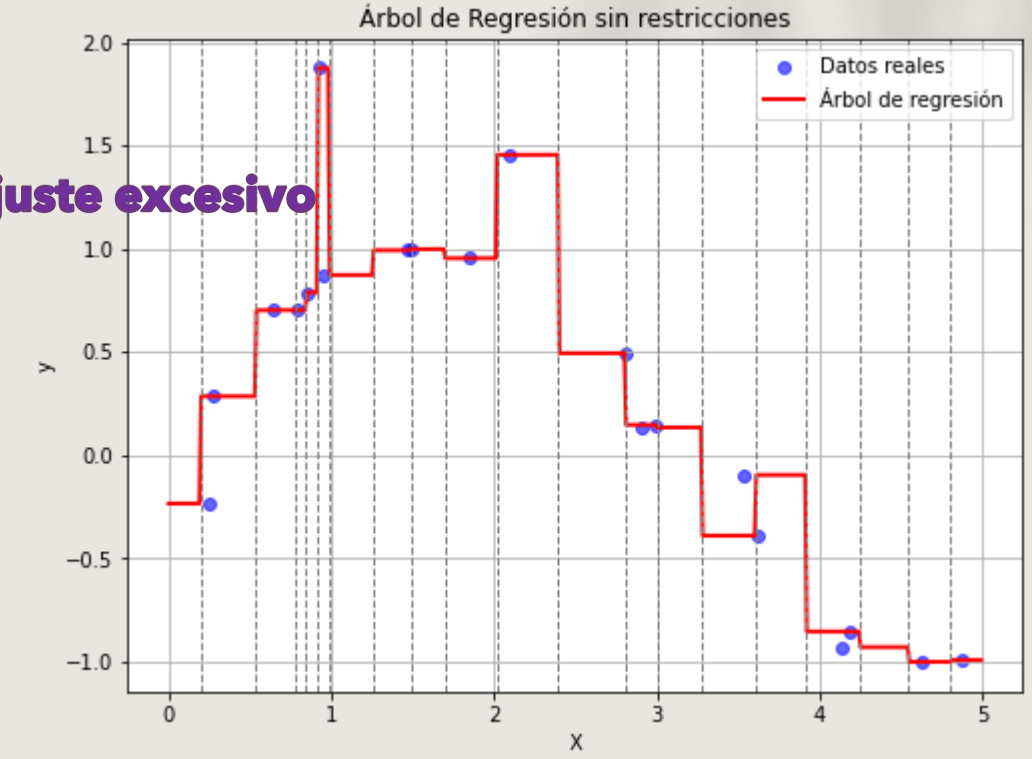

Estructura del Árbol de Regresión
X <= 3.273
squared_error = 0.665
samples = 20
value = 0.296
True
False
X <= 0.535
squared_error = 0.28
samples = 14
value = 0.727
X <= 3.911
squared_error = 0.119
samples = 6
value = -0.709
X <= 0.197
squared_error = 0.067
samples = 2
value = 0.027
X <= 2.392
squared_error = 0.221
samples = 12
value = 0.844
X <= 3.6
squared_error = 0.022
samples = 2
value = -0.241
X <= 4.542
squared_error = 0.003
samples = 4
value = -0.943
squared_error = 0.0
samples = 1
value = -0.233
squared_error = -0.0
samples = 1
value = 0.286
squared_error = 0.133
samples = 9
value = 1.038
squared_error = 0.028
samples = 3
value = 0.259
squared_error = 0.0
samples = 1
value = -0.388
squared_error = -0.0
samples = 1
value = -0.095
squared_error = 0.001
samples = 2
value = -0.89
squared_error = 0.0
samples = 2
value = -0.995

Función de coste para el algoritmo CART

$$J\left(k, t_k\right)=\frac{m_{\text{left}}}{m} \text{MSE}_{\text{left}}+\frac{m_{\text{right}}}{m} \text{MSE}_{\text{right}} \quad \text{where} \begin{cases}\text{MSE}_{\text{node}}=\sum_{i \in \text{node}}\left(\hat{y}_{\text{node}}-y^{(i)}\right)^2 \\ \hat{y}_{\text{node}}=\frac{1}{m_{\text{node}}} \sum_{i \in \text{node}} y^{(i)}\end{cases}$$



sobreajuste excesivo



Árboles de decisión

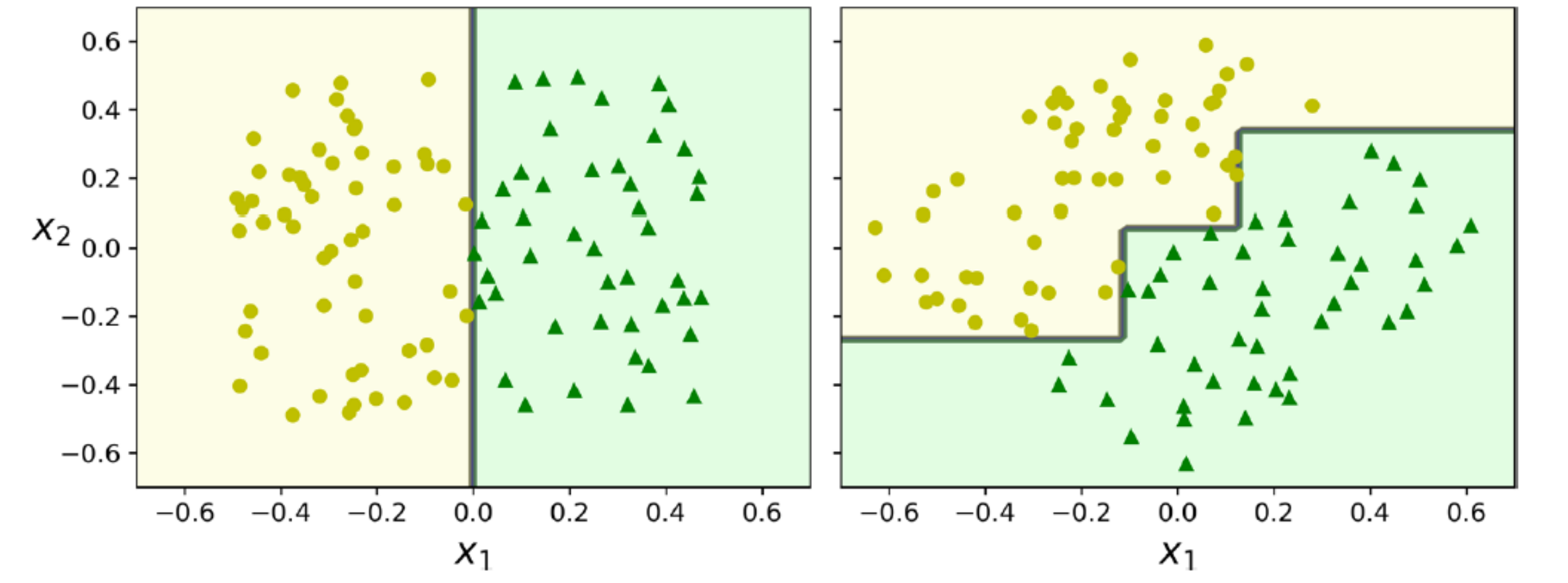
Ventajas

- Son sencillos de entender e interpretar,
- fáciles de usar,
- versátiles y potentes.

Desventajas

- los árboles de decisión funcionan mejor con límites de decisión ortogonales
- sensibilidad a pequeñas variaciones en los datos de entrenamiento.

Inestabilidad



Sensibles a pequeñas variaciones en los datos de entrenamiento

Se saco un dato : pétalo con 4,8 cm de largo y 1,8 cm de ancho

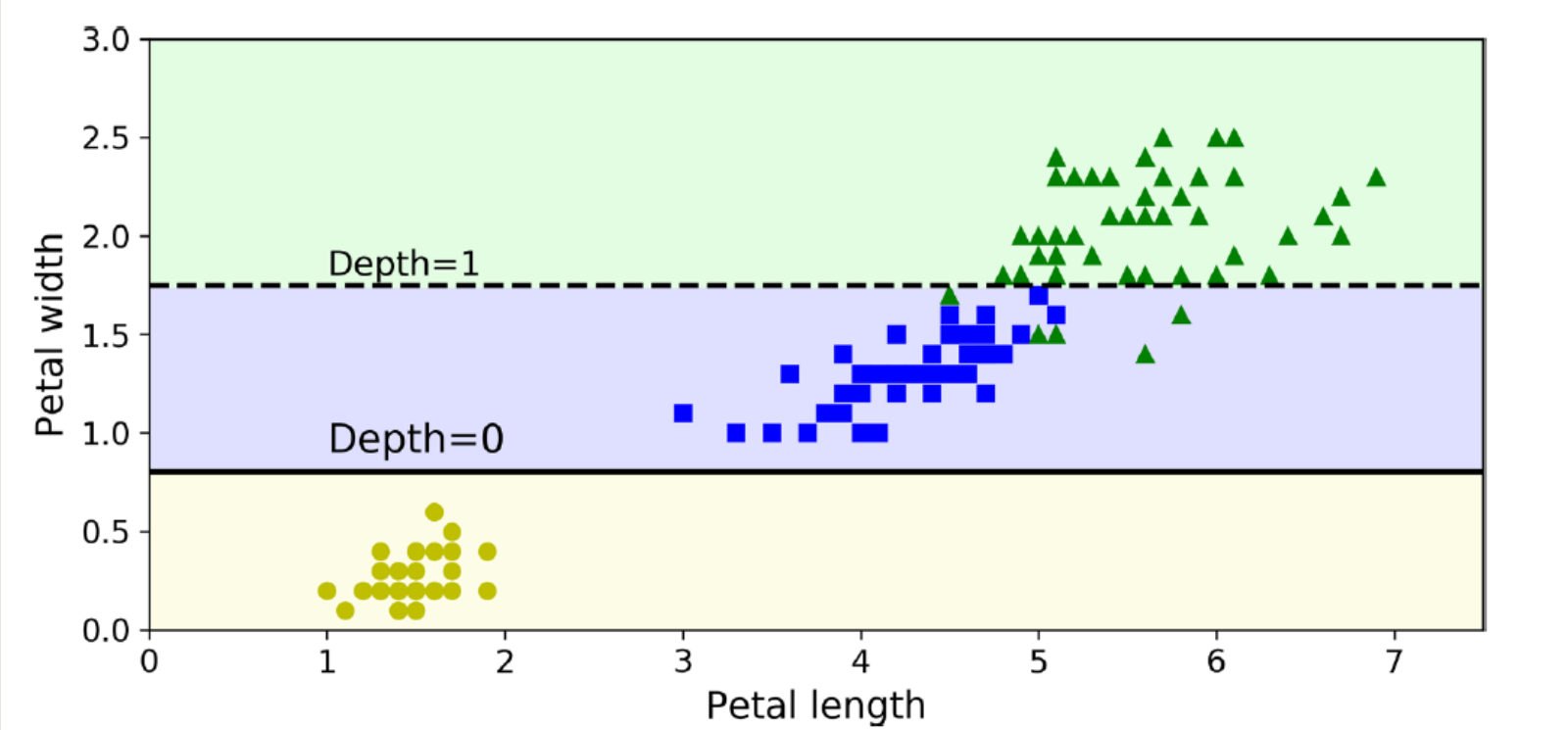


Figure 6-8. Sensitivity to training set details

La mejor manera de empezar es dejar de hablar y comenzar a actuar.

Walt Disney







Árboles de decisión

- Entrenamiento y visualización
- Efectos de los hiperparámetros en el desempeño del árbol
- Generación de predicciones



Tabla comparativa de algoritmos de árboles de decisión

Algoritmo	Criterio de división	Tipo de variables *input*	Estrategia de poda
ID3	Ganancia de información	Solo categóricas	No poda
CART	Índice de Gini	Categóricas y numéricas	Poda basada en el coste de complejidad
C4.5	Ratio de ganancia	Categóricas y numéricas	Poda basada en el error

🌲 🌲 🌲 Introducción a Random Forest

- Es un algoritmo de aprendizaje supervisado que se utiliza tanto para **clasificación** como para **regresión**.
- Fue desarrollado por Leo Breiman y Adele Cutler.
- Se basa en la idea de construir **múltiples árboles** de decisión y combinar sus resultados para mejorar la precisión y evitar el sobreajuste.

🔍 ¿Cómo funciona?

1. **Bootstrap:** Se generan múltiples subconjuntos de datos (con reemplazo) a partir del conjunto de entrenamiento original.
2. **Árboles de decisión**: Para cada subconjunto, se construye un árbol de decisión. Pero en cada nodo, en lugar de considerar todas las variables, se selecciona aleatoriamente un subconjunto de ellas.
3. **Votación o promedio**: En clasificación, cada árbol vota por una clase y se elige la clase con más votos. En regresión, se promedia la predicción de todos los árboles.

🌲 🌲 🌲 Introducción a Random Forest

🎯 Ventajas

- **Robusto al sobreajuste**: Al promediar muchos árboles, se reduce la varianza.
- **Manejo de datos faltantes y ruido**.
- **Importancia de variables**: Permite evaluar qué variables son más relevantes para la predicción.
- **Escalable**: Funciona bien con grandes volúmenes de datos.

⚠️ Desventajas

- **Menos interpretabilidad** que un solo árbol de decisión.
- **Más costoso computacionalmente**.
- Puede ser sensible a clases desbalanceadas si no se ajusta correctamente.

Gracias

Maria Laura Vranic

mlvranic@gmail.com



